

Petri Net-Based Scheduling of Single-Arm Cluster Tools With Reentrant Atomic Layer Deposition Processes

NaiQi Wu, *Senior Member, IEEE*, Feng Chu, Chengbin Chu, and MengChu Zhou, *Fellow, IEEE*

Abstract—For some wafer fabrication processes in cluster tools, e.g., atomic layer deposition (ALD), wafer revisiting is required. Typically, in such processes, wafers need to visit two consecutive processing steps several times. Such a revisiting process can be denoted as $(m_i, m_{i+1})^h$, where i means the i th-step and m_i and m_{i+1} mean the corresponding quantity of the processing modules in i and $(i + 1)$ th steps, and h the number of visiting times. This paper conducts a study for scheduling single-arm cluster tools with such a wafer revisiting process. The system is modeled by Petri nets (PNs) to guarantee the feasibility of robot activities. Based on the model, a deadlock avoidance policy is presented. With the control policy, cycle time analysis for the revisiting process is made. With the fact that wafer processing times are much longer than robot movement times in cluster tools, it is shown that, when $m_i = m_{i+1} = 1$, i.e., each step has only one processing module, the optimal one-wafer cyclic schedule is deterministic and unique, and the minimal cycle time can be calculated by an analytical expression. It is also shown that, when $m_i = 1$ and $m_{i+1} = 2$ or $m_i = 2$ and $m_{i+1} = 1$, the optimal one-wafer cyclic schedule can be obtained by finding h deterministic schedules and the one with the least cycle time. A novel analytical method is finally presented to schedule the overall system containing such reentrant wafer flow. This represents a significant advance in single-arm cluster equipment automation.

Note to Practitioners—This paper addresses the scheduling problem of single-arm cluster tools with reentrant atomic layer deposition (ALD) processes—a typical wafer revisiting pattern in semiconductor fabrication. The revisiting process is charac-

terized by two consecutive processing steps that must repeat h times. By analyzing the mechanism of such revisiting process, this paper presents a formal PN-based model and scheduling method such that an optimal one-wafer cyclic schedule can be found by comparing only h deterministic schedules. Thus, it is readily applicable to semiconductor manufacturing applications that require reentrant ADL processes.

Index Terms—Cluster tools, optimization, Petri nets (PNs), scheduling, semiconductor fabrication.

I. INTRODUCTION

AIMING at better quality control and shorter wafer fabrication lead time, semiconductor manufacturers have increasingly adopted cluster tools in their wafer fabrication processes, e.g., lithography, etching, and deposition [1], [3]. A cluster tool consists of a number of processing modules (PMs), an aligner module, a wafer handling robot, and one or two loadlocks for wafer cassette loading. The robot can be a single or dual-arm one, and the corresponding cluster tools are called single and dual-arm ones, respectively. The former is illustrated in Fig. 1, where AL and CL represent aligner and clean modules, respectively, and LA and LB represent loadlocks A and B.

Cluster tools are a kind of robotic cells. Extensive studies have been done to schedule robotic cells [2], [6]–[10], [13], [14], [20], [25], [26], [28]. These studies mainly focus on robotic flow-shop where one part type is to be processed and a completed part can stay at the machine for unlimited time. In scheduling them, the key is to obtain a cyclic schedule to maximize the throughput. A cyclic schedule is called a k -unit schedule, if exactly k parts are released into the system and at the same time k parts are completed [26]. When $k = 1$, it is a one-unit cyclic schedule. Since it is easy to understand, implement, and control, one-unit cyclic schedule is attractive in practice and is widely used in industry [10], [12]. For many practical applications, such as in wafer fabrication, the time for the robot to travel between any two distinct machines can be seen as a constant called constant robot travel time [10]. For one-unit cyclic schedule of robotic cells with a single-gripper robot and constant robot travel time, it is found that if the part processing times dominate the robot movement times, backward scheduling is optimal [2], [10]. With this result, as many as possible parts are concurrently processed.

With two loadlocks, a cluster tool can be operated in an uninterrupted way for a batch of identical wafers. Thus, a steady state cyclic schedule is desired and often it is a one-wafer cyclic schedule. To discover the operational properties of cluster tools, important effort has been made in their modeling and perfor-

Manuscript received October 08, 2009; revised January 31, 2010; accepted March 14, 2010. Date of publication April 22, 2010; date of current version January 07, 2011. This paper was recommended for publication by Associate Editor J. Yi and Editor Y. Narahari upon evaluation of the reviewers' comments. This work was supported in part by the National Science Foundation of China under Grant 60974098, Research Foundation for the Doctoral Program of Higher Education, the National 863 Program of PRC Ministry of Science and Technology under Grant 2008AA04Z109 the Ministry of Education of China, and the Chang Jiang Scholars Program, PRC Ministry of Education. This paper was presented at IEEE Conference on Automation Science and Engineering 2009, Bangalore, India, August 22–25, 2009.

N. Q. Wu is with the Department of Industrial Engineering, School of Mechanical Engineering, Guangdong University of Technology, Guangzhou 510006, China (e-mail: nqwu@gdut.edu.cn).

F. Chu is with the Laboratoire d'IBISC FRE CNRS 3190, Université d'Evry Val d'Essonne, CE 1455 Courcouronnes 91020 Evry Cédex, France (e-mail: feng.chu@ibisc.univ-evry.fr).

C. Chu is with the Laboratoire Génie Industriel, Ecole Centrale Paris, Grande voie des Vignes, Châtenay-Malabry Cedex 92295, France (e-mail: chengbin.chu@ecp.fr).

M. C. Zhou is with the Department of Electrical and Computer Engineering, New Jersey Institute of Technology, Newark, NJ 07102-1982 USA, and also with the Department of Computer Science and Technology, Tongji University, Shanghai 201804, China (e-mail: zhou@njit.edu).

Color versions of one or more of the figures in this paper are available online at <http://ieeexplore.ieee.org>.

Digital Object Identifier 10.1109/TASE.2010.2046736

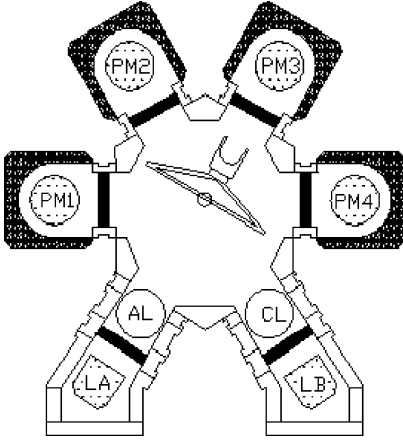


Fig. 1. Single-arm cluster tool with four processing modules.

mance evaluation [4], [5], [11], [23], [24], [29], [35], [37]. For cluster tools, the robot moving times from one PM to another can be treated as a constant and are much shorter than the wafer processing times [16]. Thus, for such cluster tools with one-wafer cyclic schedule, backward scheduling is optimal [19], [22]. This is consistent with the results obtained in [2], [10]. Therefore, single-arm cluster tools are mostly operated in this mode. With it, their modeling and performance evaluation are conducted [23], [24], [37].

Some wafer fabrication processes, such as atomic layer deposition (ALD), are repeated processes. Often, they require exactly identical processing conditions. Thus, wafers need to visit a PM multiple times [18]. With wafer revisiting, a cluster tool is no longer a flow-shop, thus, the results obtained in [2], [6]–[10], [12]–[14], [20], [25], [26], [28] and backward scheduling are not applicable. Moreover, such cluster tool is deadlock-prone [18]. It is well-known in the discrete event system community that it is highly complicated to obtain an optimal schedule and avoid deadlock at the same time. Thus, the problem is very challenging. Petri net (PN) models are developed for the problem analysis in [38]. Although the analysis results can be used as a scheduling guide, it does not present a way to avoid deadlock and find an optimal schedule. To obtain a deadlock-free and optimal schedule, another PN model is presented in [18]. Based on it, a mixed integer programming model is developed for scheduling the system. The mixed integer programming can be solved in an exact way and a deadlock-free and optimal schedule can be obtained. The cyclic schedule obtained may be a k -wafer cyclic schedule with $k \geq 1$. Nevertheless, the method is not computationally efficient because its complexity is exponential with the problem size.

An ALD process in cluster tools is a practical and typical wafer fabrication process with wafer revisiting. Deposition is one of the main processes in wafer fabrication. Presently, two technologies, ALD and chemical-vapor deposition (CVD), are often used for deposition. ALD can make film deposition with mono-atomic layer precision. Furthermore, compared with conventional CVD, ALD can easily control particle generation and uses much lower processing temperature, i.e., 100 °C–400 °C, instead of 550 °C–800 °C for CVD. Thus, ALD is increasingly adopted for deposition processes. ALD controls the film thickness by repeating the deposition cycle for a number of times

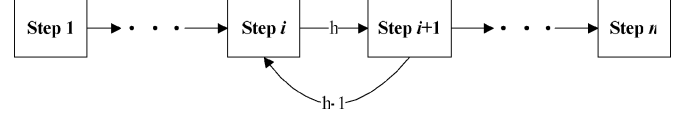


Fig. 2. The illustration of typical wafer flow pattern with revisiting.

depending on the film thickness needed, while CVD determines the thickness by adjusting the deposition time. If an efficient scheduling method can be developed, it is useful for practitioners. Furthermore, an efficient method for a special case may be useful for the further study of more general cases. Thus, in this paper, we address such a wafer flow pattern. In an ALD process, a wafer is required to visit two consecutive processing steps, say Steps i and $i+1$, several times as shown in Fig. 2. To balance the workload, there may be more than one PM in a processing step and let m_i be the number of PMs in Step i . Then, an h -time revisiting process and the wafer flow pattern can be denoted as $(m_i, m_{i+1})^h$ and $(m_1, \dots, (m_j, m_{j+1})^h, \dots, m_n)$, respectively. In an ALD process, often the number of PMs in Steps i and $i+1$ is 1 or 2. Thus, we focus on the special revisiting processes $(1, 1)^h$, $(1, 2)^h$, and $(2, 1)^h$ in this paper.

Because of its attractiveness, this work studies how to obtain a deadlock-free and optimal one-wafer cyclic schedule. Thereafter, we call a one-wafer cyclic schedule as a simple cyclic one or a cyclic one. The study done in this paper is based on the fact that wafer processing times are much longer than robot activity times in cluster tools [16].

In this paper, by using a resource-oriented model with colors and time introduced, a PN model is developed for the problem. Different from the prior ones [18], [38], it is very compact, easily extendable, and can itself eliminate all the infeasible states. Thus, the optimal schedule can be searched in a feasible space only, which greatly simplifies the scheduling complexity. Based on it, a scheduling method is presented. We show that, for revisiting pattern $(1, 1)^h$, the optimal simple cyclic schedule is unique. For $(1, 2)^h$ and $(2, 1)^h$, the optimal one can be obtained by selecting the best one among h schedules. Finally, a novel approach is presented to schedule the overall system containing such reentrant flows. With the proposed method, an optimal simple cyclic schedule can be obtained for the addressed cases by using just analytical expressions. That is much more efficient than the mixed integer programming method [18].

Next, a PN model is developed for single-arm cluster tools with revisiting. A deadlock avoidance control policy is presented in Section III by using the model. Then, schedulability and cycle time analyses are conducted in Section IV. Based on them, a method for scheduling the overall system is developed in Section V. Finally, conclusions are given in Section VI.

II. SYSTEM MODELING BY PETRI NET (PN)

In this section, we first introduce wafer revisiting patterns for a single-arm cluster tools, then develop the PN model by extending our previous work [30]–[34]. This model eliminates the infeasible robot activities such as unloading an empty module and loading an occupied module to ease the search of an optimal schedule.

We use finite capacity PN = (P, T, I, O, M, K) to model the system, where P is a finite set of places, T is a finite set

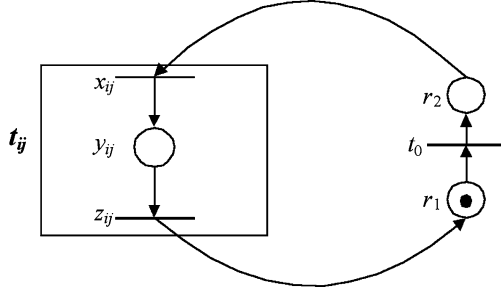


Fig. 3. A PN model for robot activities.

of transitions, $I : P \times T \rightarrow \mathbf{N}$ is an input function, where $\mathbf{N} = \{0, 1, 2, \dots\}$, $O : P \times T \rightarrow \mathbf{N}$ is an output function, $M : P \rightarrow \mathbf{N}$ is a marking with $M(p)$ being the token count in place p , and $K : P \rightarrow \{1, 2, 3, \dots\}$ is a capacity function with $K(p)$ being the maximum number of tokens that p can have [15], [30], [31], [36]. Its *preset* is the set of all input places to t , i.e., $\bullet t = \{p \in P : I(p, t) > 0\}$. Its *postset* is the set of all output places from t , i.e., $t\bullet = \{p \in P : O(p, t) > 0\}$. Similarly, $\bullet p = \{t \in T : O(p, t) > 0\}$ and $p\bullet = \{t \in T : I(p, t) > 0\}$.

For a finite capacity PN, a transition $t \in T$ in a PN is enabled if $\forall p \in P$

$$M(p) \geq I(p, t) \quad (2.1)$$

and

$$K(p) \geq M(p) - I(p, t) + O(p, t). \quad (2.2)$$

If a transition is enabled, it can fire. Firing an enabled transition t at marking M yields

$$M'(p) = M(p) - I(p, t) + O(p, t). \quad (2.3)$$

In other words, firing a transition t in finite capacity PN requires that there are enough tokens in t 's input places and enough free spaces in t 's output places.

A. Modeling Robot Activities

From the viewpoint of a robot activity, a loadlock can be seen as a PM. It is treated as the PM in Step 0. Thus, the robot activities include unloading a wafer from a PM, moving from a PM to another with a wafer, loading a wafer into a PM, moving from a PM to another without carrying a wafer, and waiting. When the robot unloads a wafer from a PM, it must move to a designated PM and loads this wafer into that PM. To eliminate infeasible robot activities, these three activities are modeled together by a macro transition t_{ij} , as shown in Fig. 3, where transition x_{ij} is for the robot unloading a wafer from a PM at Step i , place y_{ij} for moving from Step i to Step j , and transition z_{ij} for loading the wafer into a PM at Step j . We use place r_1 to model the availability of the robot. When there is a token in it, the robot is available. Because the robot move time from one PM to another can be seen as a constant [10], [16], a single transition t_0 is used to model the robot moving from any PM_i to any other PM_j without carrying a wafer. When $i = j$, it means that, after the robot loads a wafer into PM_i , it stays at PM_i and waits for the wafer to be completed. In other words, t_0 also models the

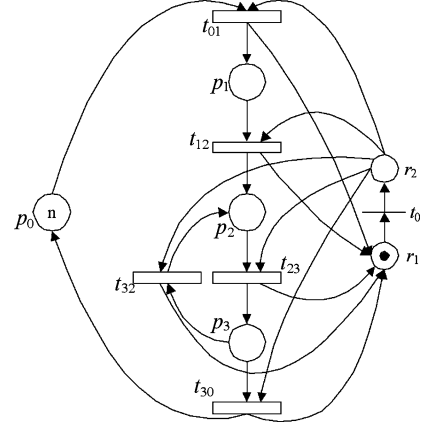


Fig. 4. The PN model for wafer flow pattern $(m_1, (m_2, m_3)^h)$.

virtual activity of the robot's moving from PM_i to PM_j . Place r_2 represents waiting before firing t_{ij} .

B. Modeling Wafer Flow

The problem here is how to model the h -time revisiting process with h being a parameter. To solve this problem, we use a resource-oriented model. The set of manufacturing resources (PMs) for Step i is modeled by a single place p_i with its capacity $K(p_i) = m_i$, which means that at most m_i tokens can be held in p_i . A token in p_i represents that a wafer is being processed by a PM in Step i . It is assumed that there are always wafers in the loadlocks to be processed. Thus, the loadlocks are modeled by place p_0 with $K(p_0) = \infty$. Then, the wafer flow can be modeled as follows. Macro transitions t_{01} (with one arc from p_0 to t_{01} and another from t_{01} to p_1) and t_{n0} are used to connect p_0 and p_1, p_n and p_0 , respectively. Macro-transition t_{ij} is put between p_i and p_j to connect p_i and p_j , if the wafers go to p_j after being processed by p_i . For revisiting process $(m_i, m_{i+1})^h$, there is a macro-transition $t_{(i+1)i}$ that connects p_{i+1} and p_i . All the activities modeled by any macro transition t_{ij} should be performed by the robot. Thus, for every transition t_{ij} , add an arc from r_2 to t_{ij} and another arc from t_{ij} to r_1 . In this way, a PN model for wafer flow with revisiting in cluster tools is obtained. The PN model for wafer flow pattern $(m_1, (m_2, m_3)^h)$ is shown in Fig. 4.

From Fig. 4, we know that for each step only one place is needed. Thus, n steps require n such places. Also, there is only a single macro-transition t_{ij} from p_i to p_j . Hence, we need only a few macro-transitions to model the system. Therefore, the model is very compact and simple. There are two simple circuits in Fig. 4: $(p_2, t_{23}, p_3, t_{32}, p_2)$ and $(p_0, t_{01}, p_1, t_{12}, p_2, t_{23}, p_3, t_{30}, p_0)$. It can be seen that the latter is not related to the revisiting process, and it is the former that describes it, which is called revisiting circuit (RVC). In Fig. 4, t_{32} can fire as many times as required, or a multiple revisiting process is well-modeled. The remaining problem is how to describe the revisiting times. This can be done by introducing colors into the model to form a colored PN.

In Fig. 4, a token in p_3 may go through t_{30} or t_{32} . When it goes through t_{32} , it is reentrant; and otherwise it is not. Thus, to describe the revisiting process, one needs to make sure in which way a token in p_3 should go. To do so, we introduce a unique color to a macro-transition in the PN.

Definition 2.1: Define the unique color of a macro-transition in the PN as $C(t_{ij}) = c_{ij}$.

Then, we define the colors for tokens. Notice that a token in r_1 enables only t_0 and there is no confusion at all. Since t_0 models the robot moving from any PM_i to any other PM_j without carrying a wafer, a token in r_2 represents that the robot is waiting for executing a transportation task associated with any t_{ij} . This reflects the flexibility in scheduling the systems. Thus, the token in r_2 enables every t_{ij} and it is not necessary to give a color to this token due to the same moving time. We need to define colors for tokens in p_i only. Based on the colors defined for transitions, we can define colors for tokens in p_i . Let W_d denote a token representing the d th wafer released into a cluster tool.

Definition 2.2: Define the color of a token W_d in p_i as $C(p_i, W_d) = c_{ij}$ if the token goes to p_j after being processed in a PM in Step i modeled by p_i .

Let $W_d(q)$ in p_i denote a token representing that the d th wafer visits p_i at the q th time. Then, if the PN shown in Fig. 4 models a wafer fabrication process that the wafers should visit the revisiting steps h times, we have $C(p_3, W_d(q)) = C(t_{32}) = c_{32}$, $1 \leq q < h$, and $C(p_3, W_d(h)) = C(t_{30}) = c_{30}$. When a token W_d has color c_{ij} , it is said that W_d enables t_{ij} . For example, token $W_d(q)$, $1 \leq q < h$, in p_3 enables t_{32} , while $W_d(h)$ in p_3 enables t_{30} . Thus, an h -time revisiting is well-modeled. We use $M(p_i)$ to denote the number of tokens in p_i regardless of the token color at marking M , and $M(p_i, c_{ij})$ to denote the number of tokens in p_i at M that have color c_{ij} .

Definition 2.3: A macro-transition t_{ij} in the PN developed above is said to be enabled at M if and only if

$$M(p_i, c_{ij}) \geq 1 \quad (2.4)$$

$$M(r_2) = 1 \quad (2.5)$$

and

$$M(p_j) < K(p_j). \quad (2.6)$$

An enabled macro-transition t_{ij} can then fire. Its firing means that its transitions x_{ij} and z_{ij} fire in a serial way. Firing x_{ij} transfers marking M to M' according to

$$M'(p_i, c_{ij}) = M(p_i, c_{ij}) - 1 \quad (2.7)$$

$$M'(r_2) = M(r_2) - 1$$

and

$$M'(r_1) = M(r_1) \quad (2.8)$$

and

$$M'(p_j) = M(p_j). \quad (2.9)$$

Firing z_{ij} transfers marking M' to M'' according to

$$M''(p_i) = M'(p_i) \quad (2.10)$$

$$M''(r_2) = M'(r_2) \quad \text{and} \quad M''(r_1) = M'(r_1) + 1 \quad (2.11)$$

and

$$M''(p_j) = M'(p_j) + 1. \quad (2.12)$$

Assume that $K(p_2) = K(p_3) = 1$ in Fig. 4. According to the above transition enabling and firing rules, t_{12} can fire at M only $M(p_1) \geq 1$, $M(r_2) = 1$, and $M(p_2) = 0$; t_{23} can fire only

TABLE I
TIME DURATIONS ASSOCIATED WITH TRANSITIONS AND PLACES

Symbol	Transition or place	Actions	Time duration
α_{ij}	x_{ij}	Robot unloads a wafer from PM_i or loadlock	α
β_{ij}	z_{ij}	Robot loads a wafer into PM_i or loadlock	β
μ_{ij}	y_{ij}	Robot moves from PM_i to PM_j with a wafer	μ
γ	t_0	Robot moves from PM_i to PM_j without a wafer	γ , when $i \neq j$ 0, when $i = j$
τ_i	p_i	Wafer is processed in p_i	τ_i
ω_{ij}	r_2	Robot waits in r_2 for firing t_{ij}	
	r_1	Robot is available	0

$M(p_2) = M(r_2) = 1$, and $M(p_3) = 0$; and t_{32} can fire only $M(p_3) = M(r_2) = 1$, $M(p_2) = 0$, and the color of the token in p_3 is c_{32} . Thereafter, when we say that a transition can fire, we mean that its enabling and firing rules are satisfied. It should be pointed out that the enabling and firing rules for a regular PN can be applied to t_0 . Thus, in this way, the wafer flow with revisiting in cluster tools is exactly described by the above developed PN.

C. Modeling Activity Times

For the purpose of scheduling analysis, it is necessary for a model to describe the temporal aspect of activities by associating time with places or transitions. In our model, both places and transitions represent some actions that take time. Thus, we associate time with both of them. We use α_{ij} and β_{ij} to denote the time for unloading a wafer from a PM (loadlock) and loading a wafer into a PM (loadlock), respectively, and μ_{ij} for the robot moving from PM_i to PM_j with a wafer. It is assumed that $\alpha_{ij} = \alpha$, $\beta_{ij} = \beta$, and $\mu_{ij} = \mu$ as done in [16]. Since the time for the robot to move from a PM to another without carrying a wafer is same, we use γ to denote the time for moving from PM_i to PM_j with $i \neq j$ as modeled by t_0 . The time is zero, when $i = j$, i.e., the virtual activity of the robot's moving from PM_i to PM_i . We use τ_i to denote the time needed for processing a wafer in p_i . A token $W_d(q)$ in p_i with color c_{ij} can enable t_{ij} only after it stays in p_i for more than τ_i time units. The time for robot waiting in r_2 before firing t_{ij} is denoted by ω_{ij} . Place r_1 is just used to model the robot availability and no activity is associated with it. Thus, it takes no time. The symbols and time durations for different transitions and places are shown in Table I.

Up to now, we complete the PN modeling of cluster tools with wafer revisiting. Based on this PN model, a deadlock-free scheduling method is developed next.

III. DEADLOCK AVOIDANCE POLICY

Based on the PN model developed in the last section, this section presents a necessary and sufficient deadlock avoidance policy such that the system can be scheduled in a feasible space.

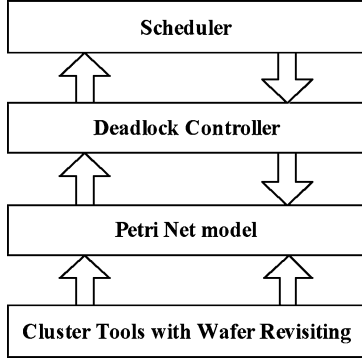


Fig. 5. Operational architecture for the scheduling problem.

In scheduling a cluster tool, effort should be made to avoid two robot activities: 1) robot unloads an empty PM and 2) robot loads an occupied PM [6], [12], [18]. In the PN model developed above, a macro-transition t_{ij} is enabled if and only if conditions (2.4) through (2.6) are met. Thus, the robot can move to Step i and then to Step j , only t_{ij} is enabled. By (2.4), it is ensured that there is a wafer at Step i that should be processed next by Step j . By (2.6), there must be at least a free space in p_j . Therefore, the two infeasible robot activities are simply avoided by the model.

Besides infeasible activities, deadlock should be avoided. Let ST_i denote the set of tokens in p_i at marking M representing the wafers being processed in Step i . For example, $ST_2 = \{W_d(2)\}$ represents that the d th wafer for its 2nd visiting is being processed in Step 2. With J steps, a marking can be equivalently represented by $(ST_1, ST_2, \dots, ST_J)$. Assume that the wafer flow pattern for a system modeled by the PN shown in Fig. 4 is $(m_1, (m_2, m_3)^2) = (1, (1, 1)^2)$ and marking $M = (\{W_3(1)\}, \{W_2(1)\}, \{W_1(1)\})$ is reached. Then, $W_3(1)$ in p_1 , $W_2(1)$ in p_2 , and $W_1(1)$ in p_3 enable t_{12} , t_{23} , and t_{32} , respectively. However, there is no free space in p_1 , p_2 , and p_3 and no macro-transition is enabled. Thus, the PN is deadlocked. Such a deadlock results from circuit wait in an RVC.

To obtain a deadlock-free schedule, two strategies can be used: 1) model and solve the deadlock avoidance and scheduling problems together as done in [18] and 2) solve these two problems in a hierarchical way by developing a deadlock avoidance policy at the lower level and solving the scheduling problem at the upper level with an architecture shown in Fig. 5. By the latter, a schedule can be searched just in the feasible space, which makes the scheduling problem significantly easier. Based on this architecture, a deadlock control policy is presented below.

According to [30] and [31], we need to avoid deadlock in RVC only in the developed model. A macro-transition t_{ij} is said to be an input (output) transition of an RVC if t_{ij} is not on it, but $t_{ij}^\bullet$ ($^\bullet t_{ij}$) is on it. Notice that an RVC in the PN model developed in the last section has one and only one input (output) transition. For example, in Fig. 4, t_{12} is the only input transition of the unique RVC and t_{30} is the only output one. Let t_{in} and t_{out} denote the input and output transitions of an RVC.

Definition 3.1: If p_i is on an RVC, $p_i \in \bullet t_{out}$, and token $W_d(q)$ in p_i at marking M enables t_{out} , this token is said to be a leaving one.

In Fig. 4, t_{out} of the unique RVC is t_{30} . Thus, assume that the wafer flow pattern for a system modeled by the PN is

$(1, (1, 1)^2)$, then token $W_d(2)$ in p_3 enabling t_{30} is a leaving one. Notice that $t_{out} = t_{30}$ and there is only one place in $\bullet t_{out}$. Thus, the leaving tokens in place $\bullet t_{out}$ are all the leaving ones of this RVC. Let $M(RVC, LEV)$ denote the number of leaving tokens of RVC at marking M . When a leaving token leaves p_i by firing t_{out} , a space in p_i is freed. Thus, a space occupied by a leaving token in p_i is potentially available. Let $P(RVC)$ denote the set of places that are on the RVC and $K(RVC) = \sum_{p_i \in P(RVC)} K(p_i)$. Further, let $M(RVC) = \sum_{p_i \in P(RVC)} M(p_i)$ be the number of tokens and $M(RVC, PAV) = K(RVC) - M(RVC) + M(RVC, LEV)$ be the number of potentially available spaces in the RVC at marking M .

Theorem 3.1: The PN model is deadlock-free if and only if for the RVC and at any marking M

$$M(RVC, PAV) \geq 1. \quad (3.1)$$

Proof: See the Appendix.

Notice that tokens are put into an RVC by firing macro-transition $t_{(i-1)i}$. This implies that the number of tokens in the RVC is controlled by the firing of t_{in} . Thus, to guarantee (3.1), we need to control t_{in} only. By this observation, we have the following result immediately.

Theorem 3.2: The PN model is deadlock-free if and only if t_{in} of each RVC is controlled such that (3.1) is satisfied at any reachable marking M .

By Theorem 3.2, if the wafer flow pattern in the revisiting process is $(m_i, m_j)^h$, we need only to keep a free space in p_i or p_j when there is no token $W_d(h)$ in p_j . When there is a token $W_d(h)$ in p_j , we can make both p_i and p_j full. Thus, Theorem 3.2 provides a simple control policy to ensure a schedule's feasibility. It should be pointed out that Theorem 3.2 is necessary and sufficient. Thus, with the model and the control policy, it excludes all infeasible schedules, but no feasible ones. Besides, there are very few places in the model, leading to much less state spaces. Hence, the PN model here significantly simplifies the scheduling problem. It should be pointed out that although the PN in [18] helps to derive a mixed integer programming, it does not reduce the searching space.

IV. CYCLE TIME ANALYSIS FOR REVISITING PROCESS

In this section, we discuss how to obtain a simple cyclic schedule of the wafer revisiting process such that the cycle time is minimized.

With backward scheduling, we simply calculate workload for individual steps and robot task load. Then, we can obtain the system cycle time. For a single-arm cluster tool with revisiting, the revisiting process $(m_i, m_{i+1})^h$ couples two steps together. Thus, workload for individual steps cannot be used to analyze the system cycle time. Furthermore, in a revisiting process $(m_i, m_{i+1})^h$, multiple schedules exist, resulting in different cycle time for a revisiting process. It is the revisiting that complicates the scheduling problem. Therefore, it is necessary to analyze the cycle time of a revisiting process and to determine how the revisiting process should be scheduled. This is done based on the PN model and deadlock avoidance control policy developed in the last two sections.

Consider Fig. 4 and assume that $(m_2, m_3)^h = (1, 1)^2$ and when t_{23} fires, a token $W_d(2)$ moves from p_2 to p_3 . At this marking, according to the deadlock control policy there are two choices to schedule the revisiting process with the transition firing sequences:

- 1) $\sigma_1 = \{ \text{wait in } r_2 \text{ for the enabling of } t_{30} \text{ (completion of } W_d(2) \text{ in } p_3) \rightarrow \text{fire } t_{30} \text{ to move } W_d(2) \text{ from } p_3 \text{ to } p_0 \rightarrow \text{fire } t_{12} \text{ to move a } W_{d+1}(1) \text{ from } p_1 \text{ into the RVC (} p_2 \text{)} \};$
- 2) $\sigma_2 = \{ \text{fire } t_{12} \text{ to move a } W_{d+1}(1) \text{ from } p_1 \text{ into the RVC (} p_2 \text{)} \rightarrow \text{wait in } r_2 \text{ for the enabling of } t_{30} \text{ (completion of } W_d(2) \text{ in } p_3) \rightarrow \text{fire } t_{30} \text{ to move } W_d(2) \text{ from } p_3 \text{ to } p_0 \}.$

With either schedule, the same marking $(\{W_{d+1}(1)\}, \{\})$ is reached. However, the processing progress for $W_{d+1}(1)$ in p_2 is different. While processing $W_{d+1}(1)$ in p_2 just begins with σ_1 , $W_{d+1}(1)$ has already been processed for some time with σ_2 . Since the wafer processing times are much longer than the robot activity times, by the second one, $W_d(2)$ in p_3 and $W_{d+1}(1)$ in p_2 are being processed concurrently. Thus, the second one is better than the first one in the sense of throughput. Notice that, in $(m_i, m_{i+1})^h$, there is no buffer, and any wafer released into the revisiting process is being processed by a PM. Hence, if we release as many wafers as possible into it, the cluster tool's concurrency is maximal and the maximal productivity can be reached. This can be stated as the following proposition.

Proposition 4.1: In order to achieve the maximal throughput of a revisiting process, it should release as many tokens (wafers) as possible to the RVC according to the deadlock control policy presented in Theorem 3.2.

Let $R_c(M_0)$ denote the set of all the markings that are reachable when the deadlock control policy given in Theorem 3.2 is applied to the PN model. Further, let $\Pi = \max_{M_i \in R_c(M_0)} (M_i(\text{RVC}))$.

Definition 4.1: A marking $M \in R_c(M_0)$ is said to be a feasible and maximally concurrent marking for a revisiting process $(m_i, m_{i+1})^h$, if $M(\text{RVC}) = \Pi$.

In a steady-state cyclic schedule for the revisiting process, it must undergo a sequence of markings and a cycle begins with a marking M and ends with the same one. In each cycle, these markings are repeated. In analyzing the cycle time, every marking undergone can be chosen as the start one. If we use ψ to denote cycle time, then $1/\psi$ is the throughput. In other words, to minimize cycle time is to maximize throughput. Thus, by Proposition 4.1, for an optimal simple cyclic schedule, the revisiting process must undergo a feasible and maximally concurrent marking M . This implies that we can select the maximally concurrent markings as the start markings for analyzing the cycle time of the revisiting process. When there is only one maximally concurrent marking, we have no choice in scheduling the revisiting process and the problem becomes easy. However, if there are multiple ones, we need to determine which results in the best schedule. In this case, the complexity is dependent on the number of such markings.

A. Revisiting Process Case One: $(1, 1)^h$

According to [18], some wafer fabrication process requires that when a wafer revisits a step, the step should be processed by the same PM. Thus, this flow pattern $(1, 1)^h$ is often applied.

When $h > 2$, the scheduling problem becomes very complicated [18].

Consider Fig. 4, the deadlock avoidance policy, and marking $M = (\{W_{d+1}(1)\}, \{W_d(h)\})$. M is feasible and $M(\text{RVC}) = 2$, and $K(\text{RVC}) = 2$. Thus, M is also a maximally concurrent marking.

Theorem 4.1: Marking $M = (\{W_{d+1}(1)\}, \{W_d(h)\})$ is the only feasible and maximally concurrent marking for a wafer revisiting process $(1, 1)^h$.

Proof: See the Appendix.

If a simple cyclic schedule undergoes the maximally concurrent marking $M = (\{W_{d+1}(1)\}, \{W_d(h)\})$, operations $W_{d+1}(1)$ and $W_d(h)$ can be processed in the most concurrent way, such that when $W_d(h)$ is completed by a PM in Step $i + 1$, the time remaining for completing $W_{d+1}(1)$ in a PM in Step i is as short as possible. Otherwise, if it does not undergo the maximally concurrent marking $M = (\{W_{d+1}(1)\}, \{W_d(h)\})$, operations $W_{d+1}(1)$ and $W_d(h)$ must be processed in a serial way. Notice that although fewer robot activities are needed by processing $W_{d+1}(1)$ and $W_d(h)$ serially, because the wafer processing times are much longer than robot activity times, it is more efficient to process $W_{d+1}(1)$ and $W_d(h)$ concurrently than serially. Thus, by Theorem 4.1, we can obtain an optimal simple cyclic schedule for a revisiting process $(1, 1)^h$ by scheduling the process from $M = (\{W_{d+1}(1)\}, \{W_d(h)\})$ to the same M . Further, it is the only choice. Hence, it is simple if such a schedule can be easily obtained.

Theorem 4.2: There is a unique and deterministic robot sequence for a simple cyclic schedule that transforms the PN model from $M = (\{W_{d+1}(1)\}, \{W_d(h)\})$ to the same M for a wafer revisiting process $(1, 1)^h$.

Proof: See the Appendix.

By combining Theorems 4.1 and 4.2, we conclude that there is only one choice for obtaining an optimal simple cycle schedule for a wafer revisiting process $(1, 1)^h$ regardless of h , i.e., the number of revisiting times. It is obvious that the presented scheduling for this case is computationally much easier than that in [18] that solves a mixed integer programming.

With the optimal simple cyclic schedule obtained for a revisiting process, we can then calculate its minimal cycle time. The optimal schedule can also be written as $\sigma_{A1} = \{(\{W_d(h)\}) \rightarrow t_0 \text{ (to } p_1) \rightarrow t_{12} \rightarrow (\{W_{d+1}(1)\}, \{W_d(h)\}) \rightarrow t_0 \text{ (to } p_3) \rightarrow t_{30} \rightarrow (\{W_{d+1}(1)\}, \{\}) \rightarrow t_0 \text{ (to } p_2) \rightarrow t_{23} \rightarrow (\{W_{d+1}(1)\}) \rightarrow \{t_{32} \text{ and then } t_{23}\} ((h - 1) \text{ times}) \rightarrow (\{W_{d+1}(h)\})\}$. We use $\theta(\bullet)$ to denote the time needed for completing a sequence of transition firings. For example, $\sigma = \{t_0 \text{ (to } p_1) \rightarrow t_{12}\}$ denotes the robot activities for firing t_0 and t_{12} , and $\theta(\sigma)$ denotes the time needed for firing them. Notice that the start time of $\sigma_1 = \{t_0 \text{ (to } p_1)\}$ and $\sigma_2 = \{\text{processing of } W_d(h) \text{ in } p_3\}$ is same, or $\sigma_3 = \{t_0 \text{ (to } p_1) \rightarrow t_{12} \rightarrow t_0 \text{ (to } p_3)\}$ and $\sigma_2 = \{\text{processing of } W_d(h) \text{ in } p_3\}$ are concurrently being done. When $\sigma_4 = \{t_0 \text{ (to } p_1) \rightarrow t_{12}\}$ is completed, then $\sigma_5 = \{\text{processing of } W_{d+1}(1) \text{ in } p_2\}$ begins. Meanwhile, after $\sigma_2 = \{\text{processing of } W_d(h) \text{ in } p_3\}$ and then $\sigma_6 = \{t_{30} \rightarrow t_0 \text{ (to } p_2)\}$ are completed, the robot reaches p_2 and can unload $W_{d+1}(1)$ in it. At this time, $W_{d+1}(1)$ in p_2 must have been completed if processing time

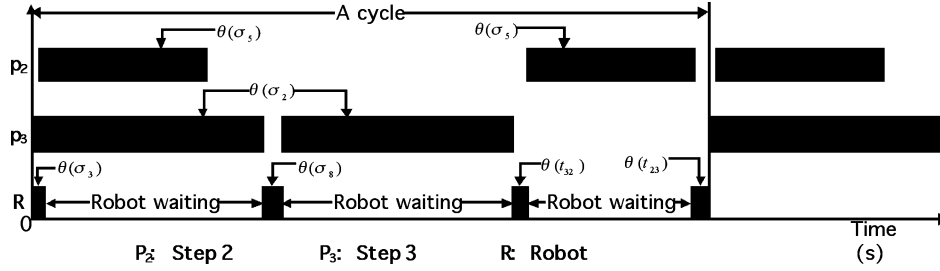


Fig. 6. Gantt chart of wafer revisiting process $(1, 1)^2$.

of $W_d(h)$ in p_3 is greater than or equal to that of $W_{d+1}(1)$ in p_2 , because $\theta(\sigma_4 = \{t_0 \text{ (to } p_1) \rightarrow t_{12}\}) = \theta(\sigma_6 = \{t_{30} \rightarrow t_0 \text{ (to } p_2)\})$. Otherwise, the robot should wait at p_2 for the completion of $W_{d+1}(1)$. For $\sigma_7 = \{t_{32} \rightarrow t_{23}\}$, before t_{32} can fire, the robot should wait for the completion of $W_{d+1}(1)$ in p_3 from the beginning of processing $W_{d+1}(1)$. The same situation applies to t_{23} . Notice that after $\sigma_8 = \{t_0 \text{ (to } p_2) \rightarrow t_{23} \rightarrow (\{\}, \{W_{d+1}(1)\})\}$ (wafer $W_{d+1}(1)$ is put in p_3) is completed, the robot just waits for the completion of $W_{d+1}(1)$ in p_3 without moving, or a virtual firing of t_0 from p_3 to p_3 . It takes zero time units. This means that in $\sigma_9 = \{t_{32} \text{ and then } t_{23}\}$, the time for the robot's movement is zero. Thus, we have $\theta(\sigma_9 = \{t_{32} \text{ and then } t_{23}\}) = \theta(\sigma_2 = \{\text{processing } W_d(h) \text{ in } p_3\}) + \theta(\sigma_5 = \{\text{processing of } W_{d+1}(1) \text{ in } p_2\}) + \theta(\sigma_{10} = \{t_{32}\}) + \theta(\sigma_{11} = \{t_{23}\})$. Therefore, the minimal cycle time for wafer revisiting process $(1, 1)^h$ is $\psi = \max\{\theta(\sigma_3 = \{t_0 \text{ (to } p_1) \rightarrow t_{12} \rightarrow t_0 \text{ (to } p_3)\})$, $\theta(\sigma_2 = \{\text{processing } W_d(h) \text{ in } p_3\})$, $\theta(\sigma_5 = \{\text{processing of } W_{d+1}(1) \text{ in } p_2\})\} + \theta(\sigma_{12} = \{t_{30} \rightarrow t_0 \text{ (to } p_2) \rightarrow t_{23}\}) + (h - 1)\theta(\sigma_7 = \{t_{32} \rightarrow t_{23}\})$. With $\theta(\{t_{ij}\}) = (\alpha + \mu + \beta)$, $\theta(\{t_0\}) = \gamma$ or 0, $\theta(\sigma_5 = \{\text{processing of } W_{d+1}(1) \text{ in } p_2\}) = \tau_2$, and $\theta(\sigma_2 = \{\text{processing } W_d(h) \text{ in } p_3\}) = \tau_3$, and notice that all virtual t_0 's are in $\sigma_7 = \{t_{32} \rightarrow t_{23}\}$, we have

$$\begin{aligned} \psi = & \max\{(2\gamma + \alpha + \mu + \beta), \tau_2, \tau_3\} \\ & + (2(\alpha + \mu + \beta) + \gamma) \\ & + (h - 1)(2(\alpha + \mu + \beta) + \tau_2 + \tau_3). \end{aligned} \quad (4.1)$$

To show the result, a Gantt chart is given in Fig. 6 with $\tau_3 > \tau_2$. With it we have $\psi = \max\{\theta(\sigma_3), \theta(\sigma_5), \theta(\sigma_2)\} + \theta(\sigma_8) + \theta(\sigma_2) + \theta(t_{32}) + \theta(\sigma_5) + \theta(t_{23}) = \tau_3 + (2(\alpha + \mu + \beta) + \gamma) + \tau_3 + (\alpha + \mu + \beta) + \tau_2 + (\alpha + \mu + \beta) = 2\tau_3 + \tau_2 + 4(\alpha + \mu + \beta) + \gamma$. That is consistent with (4.1). When $\tau_2 > \tau_3$, $\sigma_8 = \{t_0 \text{ (to } p_2) \rightarrow t_{23} \rightarrow (\{\}, \{W_{d+1}(1)\})\}$ should be changed into $\sigma_8 = \{t_0 \text{ (to } p_2) \rightarrow \text{waiting for the completion of } W_{d+1}(1) \text{ in } p_2 \rightarrow t_{23} \rightarrow (\{\}, \{W_{d+1}(1)\})\}$. Then, $\max\{\theta(\sigma_3), \theta(\sigma_5), \theta(\sigma_2)\} = \tau_2$ but not τ_3 . The following example that is an ALD process adopted from [18] is used to show the schedule.

Example 1: A wafer revisiting process is $(1, 1)^2$, $\tau_2 = 200$ s, $\tau_3 = 250$ s, and $\alpha = \mu = \beta = \gamma = 3$ s. The optimal simple schedule for this revisiting process is $\sigma_A = \{(\{W_{d+1}(1)\}, \{W_d(2)\}) \rightarrow t_0 \text{ (to } p_3) \rightarrow t_{30} \rightarrow (\{W_{d+1}(1)\}, \{\}) \rightarrow t_0 \text{ (to } p_2) \rightarrow t_{23} \rightarrow (\{\}, \{W_{d+1}(1)\}) \rightarrow (\text{virtual } t_0) \rightarrow t_{32} \rightarrow (\{W_{d+1}(2)\}, \{\}) \rightarrow (\text{virtual } t_0) \rightarrow t_{23} \rightarrow (\{\}, \{W_{d+1}(2)\}) \rightarrow t_0 \text{ (to } p_1) \rightarrow t_{12} \rightarrow$

$(\{W_{d+2}(1)\}, \{W_{d+1}(2)\})\}$ and the minimal cycle time is calculated by (4.1) as $\psi = 739$ s, which is same as that obtained in [18]. Change τ_2 and τ_3 into $\tau_2 = 200$ s, $\tau_3 = 200$ s, the schedule is not changed, but the minimal cycle time becomes $\psi = 639$ s. Our simulation also verifies the result. Notice that the integer programming method in [18] finds a k -wafer cyclic schedule with $k \geq 1$. This implies that for this example the simple cyclic schedule is not worse than a k -wafer cyclic schedule with $k > 1$.

Example 2: Suppose that the revisiting process is $(1, 1)^3$ with the same parameters in Example 1. Then, by using (4.1) we have $\psi = 1207$ s. It is also verified by simulation.

B. Revisiting Process Case Two: $(1, 2)^h$

Now we consider a more complex wafer revisiting process $(m_i, m_{i+1})^h = (1, 2)^h$. First, we need to find the feasible and maximally concurrent markings for it. We have the following result.

Theorem 4.3: Set of markings $\{(\{W_{d+2}(1)\}, \{W_{d+1}(q), W_d(h)\}), q \in \{1, \dots, h\})\}$ forms a set of all the feasible and maximally concurrent markings for the revisiting process $(m_2, m_3)^h = (1, 2)^h$.

Proof: It can be shown in a way similar to that of Theorem 4.1 and is omitted. ■

From Theorem 4.3, it can be seen that, with wafer revisiting process $(1, 2)^h$, there are multiple feasible and maximally concurrent markings, and the number of such markings is h . Thus, to find the optimal simple cyclic schedule, we need only to find a solution for every feasible and maximally concurrent marking.

Consider a feasible and maximally concurrent marking $(\{W_{d+2}(1)\}, \{W_{d+1}(q), W_d(h)\})$ with $1 \leq q \leq h$. Based on the PN model shown in Fig. 4, we can schedule: $\sigma = \{(\{W_{d+2}(1)\}, \{W_{d+1}(q), W_d(h)\}) \rightarrow t_0 \text{ (to } p_3) \rightarrow t_{30} \rightarrow (\{W_{d+2}(1)\}, \{W_{d+1}(q), \text{null}\}) \rightarrow t_0 \text{ (to } p_2) \rightarrow t_{23} \rightarrow (\{\}, \{W_{d+1}(q), W_{d+2}(1)\})\}$. At marking $(\{\}, \{W_{d+1}(q), W_{d+2}(1)\})$, a simple cyclic schedule can be done in different ways as follows.

- 1) $\sigma_1 = \{\text{waiting in } r_2 \text{ for the enabling of } t_{32} \text{ (completion of } W_{d+2}(1) \text{ at } p_3) \rightarrow \text{fire } t_{32} \text{ (move } W_{d+2}(2) \text{ into } p_2) \rightarrow \text{process } W_{d+2}(2) \text{ in } p_2 \rightarrow \text{fire } t_{23} \text{ (move } W_{d+2}(2) \text{ to } p_3), \text{ until } W_{d+2}(1) \text{ becomes } W_{d+2}(q) \text{ in } p_3\}$. Then, deal with $W_{d+1}(q)$ in the same way such that it becomes $W_{d+1}(h)$ in p_3 .
- 2) Deal with $W_{d+1}(q)$ first such that it becomes $W_{d+1}(q+1)$ in p_3 , followed by $W_{d+2}(1)$ such that it becomes $W_{d+2}(2)$ in p_3 . Then, $W_{d+1}(q+1)$ and $W_{d+2}(2)$ again, or deal with them in an alternately way.

- 3) Deal with $W_{d+1}(q)$ twice such that it becomes $W_{d+1}(q+2)$ in p_3 , followed by $W_{d+2}(1)$ twice such that it becomes $W_{d+2}(3)$ in p_3 . Then, treat $W_{d+1}(q+2)$ twice and $W_{d+2}(3)$ twice again. This process is repeated in an alternately way till $W_{d+1}(q)$ becomes $W_{d+1}(h)$ and $W_{d+2}(1)$ becomes $W_{d+2}(q)$.

Again, notice that the wafer processing times are much longer than the robot activity times. The second one is better than the first one because two wafers can be concurrently processed although there are some extra robot movements. Thus, to obtain an optimal schedule, the second way should be applied but not the first one. Now, we compare the third one with the second one. Since the time taken for processing $W_{d+1}(q+1)$ in p_3 is τ_3 , the same as that taken for processing $W_{d+2}(1)$ in p_3 , when $W_{d+1}(q+1)$ in p_3 is completed, $W_{d+2}(1)$ in p_3 must be completed by at least more than τ_2 time units earlier. Thus, it is less concurrently processed by using the third one than that by using the second one. In the third one, we can change twice to three times or more. By doing so, it must become worse. In fact, with the increase of repeated times, it finally does the same as the first one. Hence, the second one is the best one.

Consider marking $(\{W_{d+2}(1)\}, \{W_{d+1}(q), W_d(h)\})$. Let $\varphi = q - 1$ and $\eta = h - q$, and assume that $\varphi < \eta$. Then, based on the PN model in Fig. 4 and following the scheduling method discussed above, from marking $(\{W_{d+2}(1)\}, \{W_{d+1}(q), W_d(h)\})$, for a simple cyclic schedule, the wafer revisiting process can be scheduled as follows. $\sigma_B = (\{W_{d+2}(1)\}, \{W_{d+1}(q), W_d(h)\}) \rightarrow t_0$ (to p_3) $\rightarrow t_{30} \rightarrow (\{W_{d+2}(1)\}, \{W_{d+1}(q), \text{null}\}) \rightarrow t_0$ (to p_2) $\rightarrow t_{23} \rightarrow (\{\}, \{W_{d+1}(q), W_{d+2}(1)\}) \rightarrow [t_0$ (in p_3) $\rightarrow t_{32}$ (with $W_{d+1}(q)$) $\rightarrow t_{23}$ (with $W_{d+1}(q+1)$) $\rightarrow t_0$ (in p_3) $\rightarrow t_{32}$ (with $W_{d+2}(1)$) $\rightarrow t_{23}$ (with $W_{d+2}(2)$)] (do φ times) $\rightarrow (\{\}, \{W_{d+1}(q+\varphi), W_{d+2}(q)\}) \rightarrow [t_0$ (in p_3) $\rightarrow t_{32}$ (with $W_{d+1}(q+\varphi)$) $\rightarrow t_{23}$ (with $W_{d+1}(q+\varphi+1)$)] (do $\eta - \varphi$ times) $\rightarrow (\{\}, \{W_{d+1}(h), W_{d+2}(q)\}) \rightarrow t_0$ (to p_1) $\rightarrow t_{12} \rightarrow (\{W_{d+3}(1)\}, \{W_{d+1}(h), W_{d+2}(q)\})$. In this case, it is the only best way to schedule the system. For $\eta = \varphi$ and $\varphi > \eta$, we can obtain a unique schedule similarly. Therefore, we have the following result immediately.

Theorem 4.4: For a single-arm cluster tool with wafer revisiting process $(1, 2)^h$, to obtain an optimal simple cyclic schedule from each feasible and maximally concurrent marking, we need to compute and consider only a unique and deterministic schedule.

With Theorem 4.4, we need to compute deterministic schedules for all h feasible and maximally concurrent markings and their minimal cycle times. Then, we select the best one among them. When a schedule is given, an analytical expression can be derived to calculate the minimal cycle time. Below, an example is presented to show how to do so.

Example 3: [18]: The wafer revisiting process is $(1, 2)^2$, $\tau_2 = 200$ s, $\tau_3 = 250$ s, and $\alpha = \mu = \beta = \gamma = 3$ s. It follows from [18] that there are $12! = 479\,001\,600$ possible robot sequences for the system with flow pattern $(m_1, (m_2, m_3)^2) = (1, (1, 2)^2)$. Thus, it is a very complex scheduling problem. There are two feasible and maximally concurrent markings $(\{W_3(1)\}, \{W_2(1), W_1(2)\})$ and $(\{W_3(1)\}, \{W_2(2), W_1(2)\})$.

For the schedule obtained from $(\{W_3(1)\}, \{W_2(1), W_1(2)\})$, the cycle time can be calculated by

$$\psi_C = \max\{(2\gamma + \alpha + \mu + \beta), \tau_2, \tau_3\} + 4(\alpha + \mu + \beta) + 2\gamma + \tau_2. \quad (4.2)$$

For the schedule obtained from $(\{W_3(1)\}, \{W_2(2), W_1(2)\})$, the cycle time can be calculated by

$$\psi_D = \max\{(2\gamma + \alpha + \mu + \beta), \tau_2\} + 4(\alpha + \mu + \beta) + \gamma + \tau_2 + \tau_3. \quad (4.3)$$

For the derivation of (4.2) and (4.3), please see Appendix. It follows from (4.2) and (4.3) that $\psi_C = 492$ s and $\psi_D = 689$ s. Thus, the simple cyclic schedule from marking $(\{W_3(1)\}, \{W_2(1), W_1(2)\})$ is the best one among the simple cyclic schedules. Our simulation verifies the result obtained. A two-wafer cyclic schedule is obtained in [18] by using mixed integer programming method with cycle time being 490 s. This means that the optimal two-wafer cyclic schedule is two seconds (0.4%) better than the optimal simple cyclic schedule for this case.

Change τ_2 and τ_3 into $\tau_2 = 200$ s, $\tau_3 = 200$ s, we have $\psi_C = 442$ s and $\psi_D = 639$ s. By simulation, the same result is obtained. Again, the schedule from marking $(\{W_3(1)\}, \{W_2(1), W_1(2)\})$ is the best one. This result is same as that obtained in [18], i.e., in this case, the optimal simple cyclic schedule is better than optimal k -wafer cyclic schedule with $k > 1$.

For wafer revisiting process $(2, 1)^h$, we can show that there are also h feasible and maximally concurrent markings, $(\{W_{d+2}(1), W_{d+1}(q)\}, \{W_d(h)\})$, $q \in \{1, \dots, h\}$. Similar results to wafer revisiting process $(1, 2)^h$ can be obtained for a simple cyclic schedule. We test such a process with $\tau_2 = 200$ s, $\tau_3 = 250$ s, and $\alpha = \mu = \beta = \gamma = 3$ s, the cycle time for the optimal simple cyclic schedule is 542 s that is same as that obtained in [18]. Change τ_2 and τ_3 into $\tau_2 = 200$ s, $\tau_3 = 200$ s, the cycle time for the optimal simple cyclic schedule is 442 s that is also same as that obtained in [18]. Again, this implies that the optimal simple cyclic schedule is better than optimal k -wafer cyclic schedule with $k > 1$ for this example.

V. SCHEDULING OF OVERALL SYSTEM

After the analysis for the revisiting process in the last section, we can discuss the scheduling of the overall system. We use a system with wafer flow pattern $(m_1, (m_2, m_3)^h, m_4)$ to discuss this scheduling problem. The result can be extended to any wafer fabrication process containing only one revisiting process $(m_i, m_{i+1})^h$ in its sequence. The PN model for such a system is shown in Fig. 7, where p_4 models Step 4.

With backward scheduling for single-arm cluster tools without wafer revisiting, we calculate the cycle time (workload) for every individual step and the robot cycle time. Among them, the largest one is the cycle time for the overall system. The cycle time for Step i can be calculated as [24]

$$\xi_i = \frac{\tau_i + 2(\alpha + \mu + \beta) + \gamma}{m_i}. \quad (5.1)$$

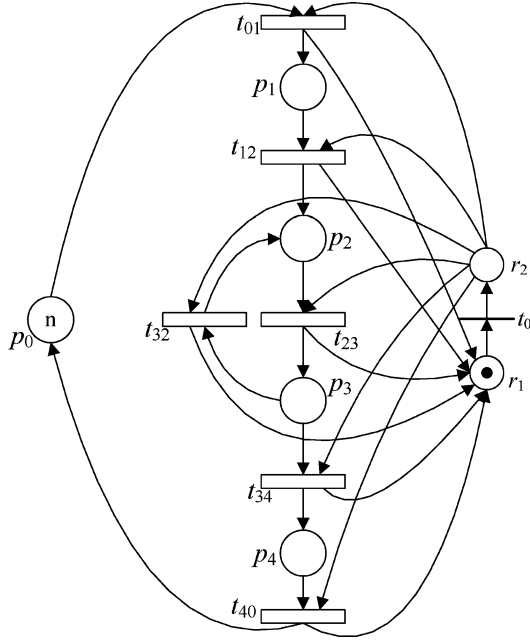


Fig. 7. The PN model for a system with wafer flow pattern $(m_1, (m_2, m_3)^h, m_4)$.

We used ξ_1 and ξ_4 to denote the cycle times of the non-revisiting Steps 1 and 4, respectively and ψ to denote the minimal cycle time of the revisiting process $(m_2, m_3)^h$. In a single-arm cluster tool with wafer revisiting, a wafer visits Step 1 first, then, enters the revisiting process. After that, the wafer leaves the revisiting process and goes into Step 4. Finally, it comes back to a loadlock. Thus, if we treat the revisiting process just as a macro step, the system is a flow line without revisiting. Let Ω denote the cycle time for the overall system and

$$\Omega \geq \max(\xi_1, \psi, \xi_4) \quad (5.2)$$

Then, we have the following result.

Theorem 5.1: If wafer flow pattern is $(m_1, (m_2, m_3)^h, m_4)$ for a single-arm cluster tool with wafer revisiting, (5.2) must hold.

To treat the revisiting process as a macro-step, we examine the optimal schedule for the revisiting process. A Gantt chart for a revisiting process $(1, 1)^2$ given in Example 1 is shown in Fig. 8, where P_2, P_3 , and R stand for Steps 2, 3, and robot, respectively. In Fig. 8, $A_1 \dots A_7$ are time points and can be calculated as: $A_1 = (\alpha + \mu + \beta) + 2\gamma$; $A_2 = \tau_3$; $A_3 = \max\{[A_2 + (\alpha + \mu + \beta) + \gamma], [\tau_2 + (\alpha + \mu + \beta) + \gamma], [A_1 + (\alpha + \mu + \beta) + \gamma]\} + (\alpha + \mu + \beta)$; $A_4 = A_3 + \tau_3$; $A_5 = A_4 + (\alpha + \mu + \beta)$; $A_6 = A_5 + \tau_2$; and $A_7 = A_6 + (\alpha + \mu + \beta)$. For $h > 2$, they can be calculated similarly. Based on Theorem 5.1, we can discuss how to schedule the overall system.

Definition 5.1: A scheduling process for wafer flow pattern $(m_1, (m_2, m_3)^h, m_4)$ is called pseudo-backward scheduling, p -backward scheduling in short, if it is scheduled as follows: unload a wafer from Step 1 and load it into the revisiting process $\rightarrow$ the robot moves to a loadlock $\rightarrow$ unload a wafer

from the loadlock and load into Step 1 $\rightarrow$ the robot moves to Step 4 $\rightarrow$ unload a wafer from Step 4 and load into a loadlock $\rightarrow$ the robot moves to the revisiting process and schedule the revisiting process as done in the last section $\rightarrow$ unload a wafer from Step 1 and load it into the revisiting process.

Observing Figs. 7 and 8, with Definition 5.1, we know that the robot sequence $\sigma = \{t_0 \rightarrow t_{01} \rightarrow t_0 \rightarrow t_{40} \rightarrow t_0\}$ occurs in the time interval $[A_1, A_2]$ in Fig. 8. We denote this sequence as $S_{p\text{-back}}$. It takes time $\theta(S_{p\text{-back}}) = 2(\alpha + \mu + \beta) + 3\gamma$. Let $\xi_{\max} = \max(\xi_1, \xi_4)$ and $\Lambda = A_2 - A_1$. It should be pointed out that Λ may be negative. With much longer wafer processing times than robot movement times, to maximize throughput, it should keep as many wafers as possible in process. We have the following result.

Theorem 5.2: For a single-arm cluster tool with wafer revisiting with wafer flow pattern $(m_1, (m_2, m_3)^h, m_4)$, the p -backward scheduling is optimal if one of the following conditions holds:

$$1) [2(\alpha + \mu + \beta) + 2\gamma] - \Lambda + \psi \leq \xi_{\max}, \text{ if } \psi < \xi_{\max} \quad (5.3)$$

$$2) \Lambda - [2(\alpha + \mu + \beta) + 2\gamma] \geq 0, \text{ if } \psi \geq \xi_{\max}. \quad (5.4)$$

Proof: See the Appendix.

By Theorem 5.2, if the conditions given in this theorem are satisfied, we can schedule the system by simply using the p -backward scheduling based on the optimal scheduling for the revisiting process.

Example 4: The wafer flow pattern is $(1, (1, 1)^2, 1)$, $\tau_1 = 270$ s, $\tau_2 = 200$ s, $\tau_3 = 250$ s, $\tau_4 = 230$ s, and $\alpha = \mu = \beta = \gamma = 3$ s. We have $\xi_1 = 291$ s, $\xi_4 = 251$ s, $\xi_{\max} = 291$ s, and $\psi = 739$ s. The optimal schedule for the revisiting process is shown in Fig. 8 and $A_2 - A_1 = 235$ s $> [2(\alpha + \mu + \beta) + 2\gamma] = 24$ s. Thus, p -backward scheduling can be applied and the minimal cycle time for the system is $\Omega = \psi = 739$ s.

In the above example, if the wafer flow pattern is changed into $(1, (1, 2)^2, 1)$ and the other parameters are unchanged. Then, we have $\psi = 492$ s $> \xi_{\max} = 291$ s. Based on the schedule given in the last section, we have $A_2 - A_1 = 235$ s same as that in Example 4. Thus, the system can be still scheduled by p -backward scheduling with minimal cycle time $\Omega = \psi = 492$ s. Simulation verifies the results for both cases.

Then, there is a question: if the conditions in Theorem 5.2 are not met, what is the minimal cycle time for the system and how to find the optimal schedule. Due to wafer revisiting, the work load becomes heavy, often resulting in $\psi > \xi_{\max}$. In the following discussion, we assume that $\psi > \xi_{\max}$. From Fig. 8, we notice that $W_3(1)$ should be unloaded from Step 1 at time $A_7 + \gamma$ for loading to Step 2. This means that $W_3(1)$ should be completed at $A_7 + \gamma$. Hence, the robot can move to the loadlock at A_3 and unload a wafer from the loadlock into Step 1, and then come back to Step 3, if $A_3 + [(\alpha + \mu + \beta) + \gamma] + \tau_1 \leq A_7 + \gamma$ and $A_4 - A_3 \geq (\alpha + \mu + \beta) + 2\gamma$. It is also known that wafer $W_1(2)$ is loaded into Step 4 at $A_2 + (\alpha + \mu + \beta)$ and it is completed at $A_2 + (\alpha + \mu + \beta) + \tau_4$. Or the robot can move to Step 4 and unload a wafer from Step 4 into the loadlock, and then come back to Step 2 in time interval $[A_5, A_6]$, if $A_2 + (\alpha + \mu + \beta) + \tau_4 \leq$

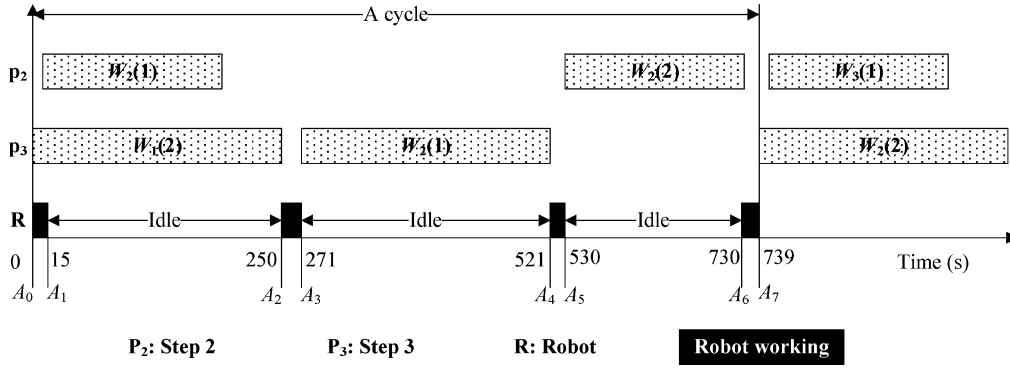
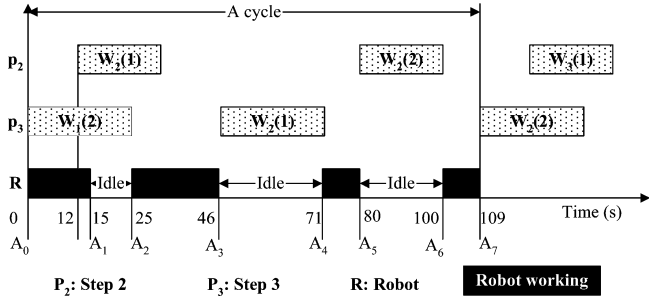
Fig. 8. The optimal simple cyclic schedule for wafer revisiting process $(1, 1)^h$.

Fig. 9. The optimal simple cyclic schedule for the revisiting process of Example 5.

$A_6 - [(\alpha + \mu + \beta) + \gamma]$ and $A_6 - [(\alpha + \mu + \beta) + 2\gamma] \geq A_5$. We define the following conditions:

$$A_3 + [(\alpha + \mu + \beta) + \gamma] + \tau_1 \leq A_7 + \gamma \quad (5.5)$$

$$A_4 - A_3 \geq (\alpha + \mu + \beta) + 2\gamma \quad (5.6)$$

$$A_2 + (\alpha + \mu + \beta) + \tau_4 \leq A_6 - [(\alpha + \mu + \beta) + \gamma] \quad (5.7)$$

$$A_6 - [(\alpha + \mu + \beta) + 2\gamma] \geq A_5. \quad (5.8)$$

Based on the above analysis, we have the following result.

Theorem 5.3: For a single-arm cluster tool with revisiting with $(m_1, (m_2, m_3)^h, m_4)$, assume that $\psi > \xi_{\max}$ and the optimal schedule for the revisiting process is as shown in Fig. 8, and the conditions in Theorem 5.2 are not satisfied. Then, the system can be scheduled such that the cycle time is ψ if one of the following conditions is satisfied.

- 1) *Condition 1:* $A_2 - A_1 \geq [(\alpha + \mu + \beta) + \gamma]$, and (5.7) and (5.8) are satisfied.
- 2) *Condition 2:* Expressions (5.5)–(5.8) are all satisfied.

Proof: See the Appendix.

Notice that the obtained schedule is not a p -backward schedule.

Example 5: The wafer flow pattern is $(1, (1, 1)^2, 1)$, $\tau_1 = 40$ s, $\tau_2 = 20$ s, $\tau_3 = 25$ s, $\tau_4 = 35$ s, and $\alpha = \mu = \beta = \gamma = 3$ s. We have $\xi_1 = 61$ s, $\xi_4 = 56$ s, $\xi_{\max} = 61$ s, and $\psi = 109$ s. The optimal simple cyclic schedule for the revisiting process is shown in Fig. 9. Notice that $A_2 - A_1 = 10 < (\alpha + \mu + \beta) + \gamma = 12$, or neither conditions in Theorem 5.2 nor Condition 1 in Theorem 5.3 are satisfied. Checking Condition 2 in Theorem 5.3, we have $A_3 + [(\alpha + \mu + \beta) + \gamma] + \tau_1 =$

$46 + 12 + 40 = 98 < A_7 + \gamma = 109 + 3 = 112$, $A_4 - A_3 = 25 > (\alpha + \mu + \beta) + 2\gamma = 15$, $A_2 + (\alpha + \mu + \beta) + \tau_4 = 25 + 9 + 35 = 69 < A_6 - [(\alpha + \mu + \beta) + \gamma] = 100 - 12 = 88$, and $A_6 - [(\alpha + \mu + \beta) + 2\gamma] = 100 - 15 = 85 > A_5 = 80$. This implies that Condition 2 in Theorem 5.3 is satisfied. Thus, we schedule the robot to pick a wafer from the loadlock to Step 1 at $A_3 = 46$ s and to unload the completed wafer at Step 4 to the loadlock at $A_5 = 80$ s. In this way, it makes no delay for any previously scheduled activity for the revisiting process. Simulation shows that this is the case.

When the conditions given in Theorem 5.3 are not satisfied, the cycle time for the system must be greater than ψ . However, the scheduling method presented by Theorem 5.3 can still be used to obtain an optimal schedule for the system, as shown in the following example.

Example 6: The wafer flow pattern is $(1, (1, 1)^2, 1)$, $\tau_1 = 40$ s, $\tau_2 = 20$ s, $\tau_3 = 25$ s, $\tau_4 = 35$ s, and $\alpha = \mu = \beta = \gamma = 4$ s. We have $\xi_1 = 68$ s, $\xi_4 = 63$ s, $\xi_{\max} = 68$ s, and $\psi = 117$ s.

The optimal schedule for the revisiting process is shown in Fig. 10(a). From Fig. 10(a) we know that: 1) neither conditions in Theorem 5.2 nor Condition 1 in Theorem 5.3 is satisfied; 2) (5.5)–(5.7) are satisfied; and 3) (5.8) is violated (see the Appendix for detail analysis). This implies that the time interval $[A_5, A_6]$ is not large enough for the robot to deliver the completed wafer from Step 4 to the loadlock. Thus, a five-second delay for delivering $W_2(2)$ from Step 2 to Step 3 is inevitable. The final schedule is shown in Fig. 10(b) and it is optimal simple cyclic schedule.

From Example 6, it is interesting to notice that, according to the conventional calculation of robot cycle time, it is 112 s that is less than $\psi = 117$ s. However, the minimal cycle time for the revisiting process is 122 s (the maximal workload for the system), neither 112 s, nor 117 s. This is caused by the revisiting process that complicates the scheduling of single-arm cluster tools.

If a cluster tool without wafer revisiting operates in a process-bound region, then the cycle time of the system is $\Omega = \max\{\xi_i, i \in \{1, 2, \dots, n\}\}$. Thus, it is important to analyze the work load ξ_i for each step. Perkinson *et al.* [24] presents a method to analyze the work load for steps to be revisited, but does not present a method to analyze the minimal cycle time. The cycle time for a cluster tool with wafer revisiting must be greater than or equal to $\Omega = \max\{\xi_i, i \in \{1, 2, \dots, n\}\}$. Thus,

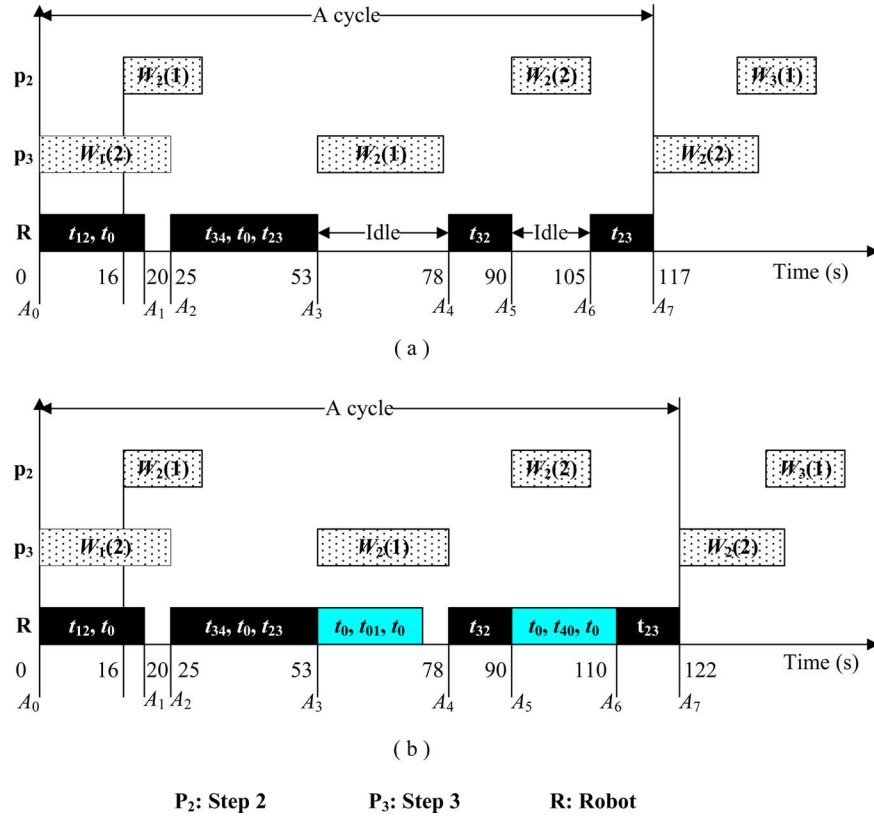


Fig. 10. The optimal schedule for Example 6.

it is different from cluster tools without wafer revisiting. Here, Ω can be used as just a lower bound of cycle time for cluster tools with wafer revisiting. For example, it is shown in [18] that $\Omega = 542$ s for the system in Example 4, but the minimal cycle time is 739 s instead of 542 s.

VI. CONCLUSION

Wafer fabrication often requires some processes, e.g., atomic layer deposition (ALD), to be repeated for several times, leading to revisiting wafer flows. With wafer revisiting, deadlock exists and the backward scheduling for single-arm cluster tools is no longer applicable. Significantly different from scheduling single-arm cluster tools with no revisiting, engineers face far more choices in sequencing robot activities and at the same time deadlock should be completely avoided. Thus, the scheduling problem becomes very complicated.

Considering a typical wafer revisiting pattern $(m_i, m_{i+1})^h$, this paper investigates the deadlock-free scheduling issue for single-arm cluster tools. Due to its easy understanding and implementation, a simple cyclic schedule (or one-wafer cyclic schedule) is pursued. With a resource-oriented PN model and control policy derived, all the infeasible schedules are eliminated so that the problem is drastically simplified. Based on the model and the fact that the wafer processing times are much longer than robot activities, cycle time and scheduling analyses are made. It is shown that, for wafer revisiting pattern $(1, 1)^h$, the optimal simple schedule is deterministic and unique, and the minimal cycle time can be calculated by an analytical expression. It is also shown that, for wafer revisiting pattern $(1, 2)^h$ or $(2, 1)^h$, the optimal simple cyclic schedule can be

obtained by finding and evaluating h deterministic schedules. Examples show that, for some cases, an optimal simple cyclic is not worse than an optimal k -wafer cyclic schedule with $k > 1$. The proposed approach is computationally efficient, especially when the number of revisiting times h is relatively large. Based on the scheduling of a revisiting process, a novel method is presented to schedule the overall system for single-arm cluster tools by treating the revisiting process as a macro-step. With the help of PN models, this work has significantly advanced the state-of-the-art in scheduling single-arm cluster tools with reentrant ALD processes.

In this paper special ALD revisiting processes are addressed. It is meaningful to generalize the results obtained to more general cases. In some wafer fabrication processes, there is wafer residency time constraint. However, in this paper, we do not consider this constraint. It is our future work to extend this work to the situation with wafer residency constraint in a wafer revisiting process. Also, the system considered in this paper is thought to be deterministic as done by most researchers in this field. However, in reality, uncertainties including failures, interruption, and time variations have significant impacts [17], [27]. Thus, such factors should be considered in scheduling cluster tools as the future work.

APPENDIX

A1: The Proof of Theorem 3.1.

Sufficiency: We need to show if (3.1) is satisfied, the corresponding RVC is deadlock-free. Assume that the wafer flow pattern in the revisiting process is $(m_i, m_j)^h$. Then, $RVC = (p_i, t_{ij}, p_j, t_{ji}, p_i)$ and $p_j \in \bullet t_{out}$. At a marking M , if for every

token $W_d(q)$ in p_j , we have $q < h$, or no token in p_j enables t_{out} . In this case, to satisfy (3.1), there must be at least a free space in p_i or p_j . Assume that there is a space in p_i and there is at least a token in p_j . Then, the tokens in p_j should visit p_i next, or t_{ji} is enabled. After firing t_{ji} , a token $W_d(q)$ goes from p_j to p_i . After staying in p_i for some time, this token $W_d(q)$ must visit p_j , or t_{ij} can fire in a finite amount of time. When $W_d(q)$ enters p_j , it may become $W_d(h)$ or not. If not, the above transition firing process can be repeated. If yes, t_{out} can fire and $W_d(h)$ leaves the RVC. When the RVC is emptied, $t_{(i-1)i}$ can fire and a token $W_d(1)$ enters p_i , and the above transition firing process can be repeated. When there is a token $W_d(h)$ in p_j and both p_i and p_j are full of tokens, (3.1) is still satisfied. In this case, t_{out} can fire to remove $W_d(h)$ from p_j . Then, a space previously occupied by $W_d(h)$ is freed. In this way, the RVC is never deadlocked.

Necessity: We need only to show that when (3.1) is violated the RVC is deadlocked. To violate (3.1), there must be no token $W_d(h)$ in p_j , and both p_i and p_j are full. In this case, there is a circuit wait in the RVC and the RVC is deadlocked. ■

A2: The Proof of Theorem 4.1.

Let the first and second revisiting steps be p_i and p_j , respectively. Because $M(RVC) = 2$, for any maximally concurrent marking M' , $M'(RVC) = 2$ must hold. Assume that M' is maximally concurrent marking such that a token in p_j is $W_d(q)$ with $q < h$. Then, according to the deadlock avoidance control policy, p_i must be empty and $M'(RVC) = 1$. This contradicts that $M'(RVC) = 2$. To reach a feasible and maximally concurrent marking, token $W_d(h)$ must first enter p_j before a token enters p_i . Then, a token can enter p_i by firing $t_{(i-1)i}$. Such a token must be $W_{d+1}(1)$. After $W_{d+1}(1)$ enters p_i , it cannot move before $W_d(h)$ leaves p_j . Hence, the theorem is proved. ■

A3: The Proof of Theorem 4.2.

For a simple cyclic schedule, in a cycle, one wafer enters and leaves the revisiting process. Then, based on Fig. 4 and deadlock avoidance policy, at $M = (\{W_{d+1}(1)\}, \{W_d(h)\})$, only t_{30} can fire. After its firing, the marking is changed to $(\{W_{d+1}(1)\}, \{\})$ at which only t_{23} can fire. Its firing changes the marking to $(\{\}, \{W_{d+1}(1)\})$ at which only t_{32} can fire. In this way, we have the following unique transition firing sequence: $\sigma_A = \{M = (\{W_{d+1}(1)\}, \{W_d(h)\}) \rightarrow t_0 \text{ (to } p_3) \rightarrow t_{30} \rightarrow (\{W_{d+1}(1)\}, \{\}) \rightarrow t_0 \text{ (to } p_2) \rightarrow t_{23} \rightarrow (\{\}, \{W_{d+1}(1)\}) \rightarrow t_{32} \text{ and then } t_{23}\} (h-1 \text{ times}) \rightarrow (\{\}, \{W_{d+1}(h)\}) \rightarrow t_0 \text{ (to } p_1) \rightarrow t_{12} \rightarrow (\{W_{d+2}(1)\}, \{W_{d+1}(h)\})\}$. ■

A4: The derivation of analytical calculation of cycle times for Example 3.

For $(\{W_3(1)\}, \{W_2(1), W_1(2)\})$, according to the scheduling method presented Section IV-B, we have the following simple cyclic schedule: $\sigma_C = (\{(\{W_3(1)\}, \{W_2(1), W_1(2)\}) \rightarrow t_0 \text{ (to } p_3) \rightarrow t_{30} \rightarrow (\{W_3(1)\}, \{W_2(1), \text{null}\}) \rightarrow t_0 \text{ (to } p_2) \rightarrow t_{23} \rightarrow (\{\}, \{W_2(1), W_3(1)\}) \rightarrow t_0 \text{ (in } p_3) \rightarrow t_{32} \rightarrow (\{W_2(2)\}, \{\text{null}, W_3(1)\}) \rightarrow t_{23} \rightarrow (\{\}, \{W_2(2), W_3(1)\}) \rightarrow t_0 \text{ (to } p_1) \rightarrow t_{12} \rightarrow (\{W_4(1)\}, \{W_2(2), W_3(1)\})\}$. Note that $\sigma = \{t_0 \text{ (in } p_3)\}$ represents that the robot moves between the PMs in p_3 . With this schedule, we can derive the cycle time expression as $\psi_C = \max\{\theta(\sigma_1 = \{t_0 \text{ (to } p_1) \rightarrow t_{12} \rightarrow t_0 \text{ (to } p_3)\})\}$, $\theta(\sigma_2 = \{\text{processing } W_1(h) \text{ in } p_3\})$, $\theta(\sigma_3 = \{\text{processing of } W_3(1) \text{ in } p_2\}) + \theta(\sigma_4 = \{t_{30} \rightarrow t_0 \text{ (to } p_2)\})$

$\rightarrow t_{23}\}) + \theta(\sigma_5 = \{t_0 \text{ (in } p_3) \rightarrow t_{32}\}) + \theta(\sigma_6 = \{\text{processing of } W_2(2) \text{ in } p_2\}) + \theta(\sigma_7 = \{t_{23}\})$, or $\psi_C = \max\{(2\gamma + \alpha + \mu + \beta), \tau_2, \tau_3\} + 4(\alpha + \mu + \beta) + 2\gamma + \tau_2$.

For $(\{W_3(1)\}, \{W_2(2), W_1(2)\})$, we have the following simple cyclic schedule: $\sigma_D = (\{(\{W_3(1)\}, \{W_2(2), W_1(2)\}) \rightarrow t_0 \text{ (to } p_3) \rightarrow t_{30} \rightarrow (\{W_3(1)\}, \{W_2(2), \text{null}\}) \rightarrow t_0 \text{ (to } p_2) \rightarrow t_{23} \rightarrow (\{\}, \{W_2(2), W_3(1)\}) \rightarrow t_{32} \rightarrow (\{W_3(2)\}, \{W_2(2), \text{null}\}) \rightarrow t_{23} \rightarrow (\{\}, \{W_2(2), W_3(2)\}) \rightarrow t_0 \text{ (to } p_1) \rightarrow t_{12} \rightarrow (\{W_4(1)\}, \{W_2(2), W_3(2)\})\}$. Notice that, for this schedule, when the robot reaches p_3 to unload $W_1(2)$, $W_1(2)$ is already completed. Thus, we have $\psi_D = \max\{\theta(\sigma_1 = \{t_0 \text{ (to } p_3) \rightarrow t_{30} \rightarrow t_0 \text{ (to } p_2)\})\}$, $\theta(\sigma_2 = \{\text{processing } W_3(1) \text{ in } p_2\}) + \theta(\sigma_3 = \{t_{23}\}) + \theta(\sigma_4 = \{\text{processing of } W_3(1) \text{ in } p_3\}) + \theta(\sigma_5 = \{t_{32}\}) + \theta(\sigma_6 = \{\text{processing of } W_3(2) \text{ in } p_2\}) + \theta(\sigma_7 = \{t_{23} \rightarrow t_0 \text{ (to } p_1) \rightarrow t_{12}\})$, or $\psi_D = \max\{(2\gamma + \alpha + \mu + \beta), \tau_2\} + 4(\alpha + \mu + \beta) + \gamma + \tau_2 + \tau_3$.

A5: The Proof of Theorem 5.2.

According to Theorem 5.1, $\Omega \geq \xi_{\max}$ if $\psi < \xi_{\max}$. By p -backward scheduling, the workloads for Steps 1 and 4 do not change. With Fig. 7, we can observe that, by p -backward scheduling, after loading $W_d(1)$ into p_2 , the robot does not move to p_3 but p_0 . This implies that, at time $A_1 - \gamma$, the robot moves to p_0 and delivers a wafer to p_1 , and then moves to p_4 and delivers a wafer into p_0 , finally comes to p_3 . It takes $[2(\alpha + \mu + \beta) + 3\gamma]$ time units. Thus, the cycle time for the revisiting process becomes $\max(\psi, [2(\alpha + \mu + \beta) + 2\gamma] - \Lambda + \psi)$. By $[2(\alpha + \mu + \beta) + 2\gamma] - \Lambda + \psi \leq \xi_{\max}$, we have $\max(\psi, [2(\alpha + \mu + \beta) + 2\gamma] - \Lambda + \psi) \leq \xi_{\max}$. Thus, $\Omega = \xi_{\max}$, i.e., the schedule is optimal if (5.3) is met.

If (5.4) is met, we have $\max\{\psi, [2(\alpha + \mu + \beta) + 2\gamma] - \Lambda + \psi\} = \psi$. Thus, $\Omega = \psi$, the minimal cycle time is reached. ■

A6: The Proof of Theorem 5.3.

If Condition 1 is satisfied, we can schedule the robot to move to a loadlock at $A_1 - \gamma$ for unloading wafer $W_3(1)$ and loading into Step 1. Then, the robot comes back to Step 3, just as the p -backward scheduling does. Because $A_2 - A_1 \geq [(\alpha + \mu + \beta) + \gamma]$, this does not delay any activity in the schedule for the revisiting process. If (5.7) and (5.8) are satisfied, we can schedule the robot to move to Step 4 at $A_6 - [(\alpha + \mu + \beta) + 2\gamma]$ for unloading wafer $W_1(2)$ and loading into the loadlock, and then the robot comes back to Step 2. This does not delay any activity in the schedule for the revisiting process either. Thus, the cycle time for the system is ψ .

For Condition 2, if (5.5) and (5.6) are satisfied, we can schedule the robot to move to the loadlock at A_3 for unloading wafer $W_3(1)$ and loading into Step 1, and then the robot comes back to Step 3. Then, the unloading process of $W_1(2)$ is scheduled to take place as done in the Condition 1 case. By doing so, no delay is made for the scheduled revisiting process. Thus, the cycle time for the system is ψ . ■

A7: The analysis of Example 6.

From Fig. 10(a), we can observe that $A_2 - A_1 = 5 < (\alpha + \mu + \beta) + 2\gamma = 20$, or neither conditions in Theorem 5.2 nor Condition 1 in Theorem 5.3 is satisfied. Checking Condition 2 in Theorem 5.3, we have $A_3 + [(\alpha + \mu + \beta) + \gamma] + \tau_1 = 53 + 16 + 40 = 109 < A_7 + \gamma = 117 + 4 = 121$, $A_4 - A_3 = 25 > (\alpha + \mu + \beta) + 2\gamma = 20$, and $A_2 + (\alpha + \mu + \beta) + \tau_4 = 25 + 12 + 35 = 72 < A_6 - [(\alpha + \mu + \beta) + \gamma] = 105 - 16 = 89$,

or (5.5)–(5.7) are satisfied. However, $A_6 - [(\alpha + \mu + \beta) + 2\gamma] = 105 - 20 = 85 < A_5 = 90$, or (5.8) is violated. This implies that although there is enough time for the robot to deliver a wafer from the loadlock to Step 1 in time interval $[A_3, A_4]$, the time interval $[A_5, A_6]$ is not large enough for the robot to deliver the completed wafer from Step 4 to the loadlock.

ACKNOWLEDGMENT

The authors thank the anonymous reviewers for their helpful comments.

REFERENCES

- [1] M. Bader, R. Hall, and G. Strasser, "Integrated processing equipment," *Solid State Technol.*, vol. 33, pp. 149–154, 1990.
- [2] N. Brauner, "Identical part production in cyclic robotic cells: Concepts, overview and open questions," *Discrete Appl. Math.*, vol. 156, pp. 2480–2492, 2008.
- [3] P. Burggraaf, "Coping with the high cost of wafer fabs," *Semiconduct. Int.*, vol. 38, pp. 45–50, 1995.
- [4] W. K. V. Chan, J. Yi, and S. Ding, "Optimal scheduling of multi-cluster tools with constant robot moving times, Part I: Two-cluster analysis," *IEEE Trans. Autom. Sci. Eng.*, 2010, to be published.
- [5] W. K. V. Chan, S. Ding, J. Yi, and D. Song, "Optimal scheduling of multi-cluster tools with constant robot moving times, Part II: Tree-like topology configurations," *IEEE Trans. Autom. Sci. Eng.*, 2010, to be published.
- [6] Y. Crama, V. Kats, J. van de Klundert, and E. Levner, "Cyclic scheduling in robotic flowshops," *Ann. Oper. Res.: Math. Ind. Syst.*, vol. 96, pp. 97–124, 2000.
- [7] Y. Crama and J. van de Klundert, "Cyclic scheduling of identical parts in a robotic cell," *Oper. Res.*, vol. 45, pp. 952–965, 1997.
- [8] M. Dawande, N. Geismar, M. Pinedo, and C. Sriskandarajah, "Throughput optimization in dual-grripper interval robotic cells," *IIE Trans.*, vol. 42, no. 1, pp. 1–15, 2010.
- [9] M. Dawande, H. N. Geismar, S. P. Sethi, and C. Sriskandarajah, *Throughput Optimization in Robotic Cells*. New York: Springer, 2007.
- [10] M. Dawande, C. Sriskandarajah, and C. S. Sethi, "On throughput maximization in constant travel-time robotic cells," *Manuf. Serv. Oper. Manage.*, vol. 4, no. 4, pp. 296–312, 2002.
- [11] S. Ding, J. Yi, and M. Zhang, "Scheduling multi-cluster tools: An integrated event graph and network model approach," *IEEE Trans. Semicond. Manuf.*, vol. 19, no. 3, pp. 339–351, 2006.
- [12] I. Drobouchevitch, S. P. Sethi, and C. Sriskandarajah, "Scheduling dual-grripper robotic cells: 1-unit cycles," *Eur. J. Oper. Res.*, vol. 171, pp. 598–631, 2006.
- [13] H. N. Geismar, L. M. A. Chan, M. Dawande, and C. Sriskandarajah, "Approximations to optimal k -unit cycles for single- and dual-grripper robotic cells," *Prod. Oper. Manage.*, vol. 17, no. 5, pp. 551–563, 2008.
- [14] H. N. Geismar, M. Dawande, and C. Sriskandarajah, "Throughput optimization in constant travel-time dual-grripper robotic cells with parallel machines," *Prod. Oper. Manage.*, vol. 15, pp. 311–328, 2006.
- [15] B. Hruz and M. C. Zhou, *Modeling and Control of Discrete Event Dynamic Systems*. London, U.K.: Springer, 2007.
- [16] J.-H. Kim, T.-E. Lee, H.-Y. Lee, and D.-B. Park, "Scheduling analysis of timed-constrained dual-armed cluster tools," *IEEE Trans. Semicond. Manuf.*, vol. 16, no. 3, pp. 521–534, 2003.
- [17] J.-H. Kim and T.-E. Lee, "Schedulability analysis of timed-constrained cluster tools with bounded time variation by an extended Petri net," *IEEE Trans. Autom. Sci. Eng.*, vol. 5, no. 3, pp. 490–503, Jul. 2008.
- [18] H.-Y. Lee and T.-E. Lee, "Scheduling single-arm cluster tools with reentrant wafer flows," *IEEE Trans. Semicond. Manuf.*, vol. 19, no. 2, pp. 226–240, 2006.
- [19] T.-E. Lee, H.-Y. Lee, and Y.-H. Shin, "Workload balancing and scheduling of a single-armed cluster tool," in *Proc. 5th APIEMS Conf.*, Gold Coast, Australia, 2004, pp. 1–15.
- [20] L. Lei and T. J. Wang, "Determining optimal cyclic hoist schedules in a single-hoist electroplating line," *IIE Trans.*, vol. 26, no. 2, pp. 25–33, 1994.
- [21] Z. W. Li and M. C. Zhou, *Deadlock Resolution in Automated Manufacturing Systems: A Novel Petri Net Approach*. New York: Springer, 2009.
- [22] T. Murata, "Petri nets: Properties, analysis and applications," *Proc. IEEE*, vol. 77, no. 4, pp. 541–580, 1989.
- [23] T. L. Perkinson, P. K. MacLarty, R. S. Gyurcsik, and R. K. Cavin, III, "Single-wafer cluster tool performance: An analysis of throughput," *IEEE Trans. Semicond. Manuf.*, vol. 7, pp. 369–373, 1994.
- [24] T. L. Perkinson, R. S. Gyurcsik, and P. K. MacLarty, "Single-wafer cluster tool performance: An analysis of the effects of redundant chambers and revisitations sequences on throughput," *IEEE Trans. Semicond. Manuf.*, vol. 9, pp. 384–400, 1996.
- [25] S. Sethi, J. Sidney, and C. Sriskandarajah, "Scheduling in dual-grripper robotic cells for productivity gains," *IEEE Trans. Robot. Autom.*, vol. 17, pp. 324–341, 2001.
- [26] S. Sethi, C. Sriskandarajah, G. Sorger, J. Blazewicz, and W. Kubiak, "Sequencing of parts and robot moves in a robotic cell," *Int. J. Flexible Manuf. Syst.*, vol. 4, pp. 331–358, 1992.
- [27] Y.-H. Shin, T.-E. Lee, J.-H. Kim, and H.-Y. Lee, "Modeling and implementing a real-time scheduler for dual-armed cluster tools," *Comput. Industry*, vol. 45, no. 1, pp. 13–27, 2001.
- [28] Q. Su and F. Chen, "Optimal sequencing of double-grripper gantry robot moves in tightly-coupled serial production systems," *IEEE Trans. Robot. Autom.*, vol. 12, pp. 22–30, 1996.
- [29] S. Venkatesh, R. Davenport, P. Foxhoven, and J. Nulman, "A steady state throughput analysis of cluster tools: Dual-blade versus single-blade robots," *IEEE Trans. Semicond. Manuf.*, vol. 10, pp. 418–424, 1997.
- [30] N. Q. Wu, "Necessary and sufficient conditions for deadlock-free operation in flexible manufacturing systems using a colored Petri net model," *IEEE Trans. Syst., Man, Cybern.*, vol. 29, no. 2, pt. Part C, pp. 192–204, May 1999.
- [31] N. Q. Wu and M. C. Zhou, "Avoiding deadlock and reducing starvation and blocking in automated manufacturing systems based on a Petri net model," *IEEE Trans. Robot. Autom.*, vol. 17, no. 5, pp. 658–669, 2001.
- [32] N. Q. Wu and M. C. Zhou, "Modeling and deadlock control of automated guided vehicle systems," *IEEE/ASME Trans. Mech.*, vol. 9, no. 1, pp. 50–57, 2004.
- [33] N. Q. Wu and M. C. Zhou, "Modeling and deadlock avoidance of automated manufacturing systems with multiple automated guided vehicles," *IEEE Trans. Syst., Man, Cybern.*, vol. 35, no. 6, pt. Part B, pp. 1193–1202, 2005.
- [34] N. Wu, M. C. Zhou, and Z. Li, "Resource-oriented Petri net for deadlock avoidance in flexible assembly systems," *IEEE Trans. Syst., Man, Cybern.: Part A*, vol. 38, no. 1, pp. 56–69, Jan. 2008.
- [35] J. Yi, S. Ding, D. Song, and M. Zhang, "Steady-state throughput and scheduling analysis of multi-cluster tools: A decomposition approach," *IEEE Trans. Autom. Sci. Eng.*, vol. 5, no. 2, pp. 321–336, Apr. 2008.
- [36] M. C. Zhou and K. Venkatesh, *Modeling, Simulation and Control of Flexible Manufacturing Systems: Petri Net Approach*. Singapore: World Scientific, 1998.
- [37] W. M. Zuberek, "Timed Petri nets in modeling and analysis of cluster tools," *IEEE Trans. Robot. Autom.*, vol. 17, no. 5, pp. 562–575, 2001.
- [38] W. M. Zuberek, "Cluster tools with chamber revisiting—Modeling and analysis using timed Petri nets," *IEEE Trans. Semicond. Manuf.*, vol. 17, no. 3, pp. 333–344, 2004.



NaiQi Wu (M'04–SM'05) received the M.S. and Ph.D. degrees in systems engineering both from Xi'an Jiaotong University, Xi'an, China in 1985 and 1988, respectively.

From 1988 to 1995, he was with the Chinese Academy of Sciences, Shenyang Institute of Automation, Shenyang, China, and from 1995 to 1998, with Shantou University, Shantou, China. From 1991 to 1992, he was a Visiting Scholar at the School of Industrial Engineering, Purdue University, West Lafayette, IN. In 1999, 2004, and 2007–2009, he was a Visiting Professor with the Department of Industrial Engineering, Arizona State University, Tempe, the Department of Electrical and Computer Engineering, New Jersey Institute of Technology, Newark, NJ, and the Industrial Systems Engineering Department, Industrial Systems Optimization Laboratory, University of Troyes, Troyes, France, respectively. He is currently a Professor of Industrial and Systems Engineering in the Department of Industrial Engineering, School of Mechatronics Engineering, Guangdong University of Technology, Guangzhou, China. He is the author or coauthor of many papers published in the *International Journal of Production Research*, the *IEEE TRANSACTIONS ON SYSTEMS, MAN, AND CYBERNETICS*, the *IEEE TRANSACTIONS ON ROBOTICS AND AUTOMATION*, the *IEEE TRANSACTIONS ON AUTOMATION SCIENCE AND ENGINEERING*, the *IEEE/ASME TRANSACTIONS ON MECHATRONICS*, the *IEEE TRANSACTIONS ON SEMICONDUCTOR MANUFACTURING*, the *Journal of Intelligent Manufacturing*, *Production Planning and Control*, and *Robotics and Computer Integrated Manufacturing*. His research interests include production planning and scheduling, manufacturing system modeling and control, discrete event systems, Petri net theory and applications, Logistics, and information assurance.

Dr. Wu is an Associate Editor of the IEEE TRANSACTIONS ON SYSTEMS, MAN, AND CYBERNETICS, PART C and the IEEE TRANSACTIONS ON AUTOMATION SCIENCE AND ENGINEERING and Editor-in-Chief of the *Industrial Engineering Journal*. He was a Program Committee Member of the 2003–2010 IEEE International Conference on Systems, Man, and Cybernetics, a Program Committee Member of the 2005–2009 IEEE International Conference on Networking, Sensing and Control, a Program Committee Member of the 2006, 2008–2010 IEEE International Conference on Automation Science and Engineering, a Program Committee Member of the 2006 IEEE International Conference on service systems and service management, a Program Committee Member of the 2007 International Conference on Engineering and Systems Management, and reviewer for many international journals.



Feng Chu received the B.S. and M.S. degrees in electrical engineering from Hefei University of Technology, Hefei, China, in 1986 and the Institut National Polytechnique de Lorraine, France, in 1991, respectively, and the Ph.D. degree in computer science from University of Metz, Metz, France, in 1995.

She was with the Jiangsu University of Technology, China, for two years and at the National Research Institute in Computer Science and Automation (INRIA), France, for four years. She was

an Associate Professor at the University of Technology, Troyes, France, from 1999 to 2009. She joined the laboratory at IBISC in 2009. She is currently a Full Professor at the Université d'Evry Val d'Essonne. She has published more than 30 papers in international journals such as the *European Journal of Operational Research*, *Computers and Operations Research*, *Applied Mathematics and Computation*, *Computers and Industrial Engineering*, the *International Journal of Computer Integrated Manufacturing*, the *International Journal of Production Research*, *Transportation Research*, the IEEE TRANSACTIONS ON SYSTEMS, MAN AND CYBERNETICS, the IEEE TRANSACTIONS ON ROBOTICS AND AUTOMATION, the IEEE TRANSACTIONS ON AUTOMATION SCIENCE AND ENGINEERING, etc., and over 40 papers in academic conferences. She is mainly interested in modeling, analysis and optimization of complex systems including intelligent transportation systems, logistic and production systems based on Petri nets and operations research.



Chengbin Chu received the B.Sc. degree in electrical engineering from Hefei University of Technology, Hefei, China, in 1985 and the Ph.D. degree in computer science from Metz University, Metz, France, in 1990.

He was with the National Research Institute in Computer Science and Automation (INRIA), France, from 1987 to 1996. He was a Professor with the University of Technology, Troyes, France, from 1996 to 2008, where he was also Founding Director of the Industrial Systems Optimization Laboratory.

He is currently a Senior Professor of the Chair Supply Chain at Ecole Centrale Paris. He was nominated Chang Jiang Scholars Program Chair Professor by the Chinese Ministry of Education in 2005. He is an Overseas Visiting Professor and Overseas Director of the Department of Industrial Engineering at Xi'an Jiaotong University. He is author or coauthor of three books and more than 100 articles in international journals such as *Operations Research*, the *SIAM Journal of Computing*, the IEEE TRANSACTIONS ON ROBOTICS AND AUTOMATION, the IEEE TRANSACTIONS ON AUTOMATION SCIENCE AND ENGINEERING, the *International Journal of Production Research*, *Naval Research Logistics*, and so on. He also published many papers in conference proceedings. He is interested in research areas related to operations research and modeling, analysis, and optimization of supply chain and production systems.

Dr. Chu received the First Prize of the Robert Faure Award in 1996. He also received the 1998 Best Transactions Paper Award from the IEEE Robotics and Automation Society for his research and application activities. He was an Associate Editor of the IEEE TRANSACTIONS ON ROBOTICS AND AUTOMATION from 2001 to 2004. He is currently an Associate Editor of the IEEE TRANSACTIONS ON AUTOMATION SCIENCE AND ENGINEERING and the IEEE TRANSACTIONS ON INDUSTRIAL INFORMATICS.



Mengchu Zhou (S'88–M'90–SM'93–F'03) received the B.S. degree in electrical engineering from Nanjing University of Science and Technology, Nanjing, China in 1983, the M.S. degree in automatic control from the Beijing Institute of Technology, Beijing, China, in 1986, and the Ph.D. degree in computer and systems engineering from the Rensselaer Polytechnic Institute, Troy, NY, in 1990.

He joined the New Jersey Institute of Technology (NJIT), Newark, NJ, in 1990, and is currently a Professor of Electrical and Computer Engineering

and the Director of Discrete-Event Systems Laboratory. He also serves as the Director of the MS in Computer Engineering Program and Area Coordinator of Intelligent Systems at NJIT. His research interests are in intelligent automation, lifecycle engineering and sustainability evaluation, Petri nets, wireless ad hoc and sensor networks, system security, semiconductor manufacturing, and energy systems. He has over 350 publications including 10 books, 150+ journal papers (majority in IEEE transactions), and 17 book-chapters. He coauthored with F. DiCesare, *Petri Net Synthesis for Discrete Event Control of Manufacturing Systems* (Kluwer Academic, Boston, MA, 1993), edited *Petri Nets in Flexible and Agile Automation* (Kluwer Academic, 1995), coauthored with K. Venkatesh, *Modeling, Simulation, and Control of Flexible Manufacturing Systems: A Petri Net Approach* (World Scientific, 1998), coedited with M. P. Fanti, *Deadlock Resolution in Computer-Integrated Systems* (Marcel Dekker, 2005), coauthored with H. Zhu, *Object-Oriented Programming in C++: A Project-Based Approach* (Tsinghua University Press, 2005), coauthored with B. Hruz, *Modeling and Control of Discrete Event Dynamic Systems* (Springer, London, U.K., 2007), and coauthored with Z. Li, *Deadlock Resolution in Automated Manufacturing Systems: A Novel Petri Net Approach* (Springer, New York, 2009). He was invited to lecture in Australia, Canada, China, France, Germany, Hong Kong, Italy, Japan, Korea, Mexico, Singapore, Taiwan, and the U.S. and served as a plenary speaker for several conferences.

Dr. Zhou is a Life Member of the Chinese Association for Science and Technology-USA and served as its President in 1999. He was the recipient of the NSF's Research Initiation Award, CIM University-LEAD Award by Society of Manufacturing Engineers, Perlis Research Award by NJIT, Humboldt Research Award for U.S. Senior Scientists, Leadership Award and Academic Achievement Award by the Chinese Association for Science and Technology-USA, Asian American Achievement Award by the Asian American Heritage Council of New Jersey, and the Distinguished Lectureship of the IEEE SMC Society. He served as an Associate Editor of the IEEE TRANSACTIONS ON ROBOTICS AND AUTOMATION from 1997 to 2000, and the IEEE TRANSACTIONS ON AUTOMATION SCIENCE AND ENGINEERING from 2004 to 2007, and currently is an Editor of the IEEE TRANSACTIONS ON AUTOMATION SCIENCE AND ENGINEERING, and Associate Editor of the IEEE TRANSACTIONS ON SYSTEMS, MAN AND CYBERNETICS: PART A and the IEEE TRANSACTIONS ON INDUSTRIAL INFORMATICS. He served as Guest-Editor for many journals including the IEEE TRANSACTIONS ON INDUSTRIAL ELECTRONICS and the IEEE TRANSACTIONS ON SEMICONDUCTOR MANUFACTURING. He was General Chair of the IEEE Conference on Automation Science and Engineering, Washington, DC, August 23–26, 2008, General Co-Chair of the 2003 IEEE International Conference on System, Man and Cybernetics (SMC), Washington, DC, October 5–8, 2003, Founding General Co-Chair of the 2004 IEEE International Conference on Networking, Sensing and Control, Taipei, March 21–23, 2004, and General Chair of the 2006 IEEE International Conference on Networking, Sensing and Control, Ft. Lauderdale, FL, April 23–25, 2006. He was Program Chair of the 1998 and 2001 IEEE International Conference on SMC and the 1997 IEEE International Conference on Emerging Technologies and Factory Automation. He organized and chaired over 80 technical sessions and served on program committees for many conferences. He is Program Committee Chair of the 2010 IEEE International Conference on Mechatronics and Automation, August 4–7, 2010, Xi'an, China. He has led or participated in 40 research and education projects with total budget of over \$10 M, funded by the National Science Foundation, the Department of Defense, Engineering Foundation, the New Jersey Science and Technology Commission, and industry. He was the Founding Chair of the Discrete Event Systems Technical Committee and Founding Co-Chair of the Enterprise Information Systems Technical Committee of the IEEE SMC Society, and Chair (founding) of the Semiconductor Manufacturing Automation Technical Committee of the IEEE Robotics and Automation Society.